Day 3 – Module 3.1

The Theory of Weights

Why Every Observation Doesn't Count Equally

SADC Regional Training Workshop

Module Overview

Learning Objectives:

1. Understand why unweighted data produces biased estimates
2. Master the "balloon" analogy for survey weighting
3. Calculate base weights from selection probabilities
4. Interpret the relationship between π and w

Duration: 45 minutes — **Narrative:** Lindiwe explains weights to Minister

The Weight of Evidence

Zambara NSO – Monday: Lindiwe reviews results. First instinct: sum raw data...

Raw Sample		**Population**
16,752 households	vs.	18.2 M households

▷ Something is wrong with simple counting...

Why Raw Counts Lie

A sample is **not** a miniature population. Each unit represents many others—*but not equally*.

Problem 1
Unequal Selection
Urban sampled at higher rates

Problem 2
Differential NR
28% urban NR vs 8% farms

Problem 3
Coverage Gaps
Hard-to-reach invisible

Key: Without weights, estimates reflect **sample**—not **population**.

The Ballot Box Analogy

Unfair Election:

- City voters: 3 ballots each
- Rural voters: 1 ballot each
- Some villages: no ballots

Raw count $\neq$ will of people

$\Rightarrow$

Fair Result:

- City ballots: count as $1/3$
- Rural ballots: count as 1
- Missing: proxy representation

Weighted count $=$ true voice

Survey weights correct for unequal "ballots."

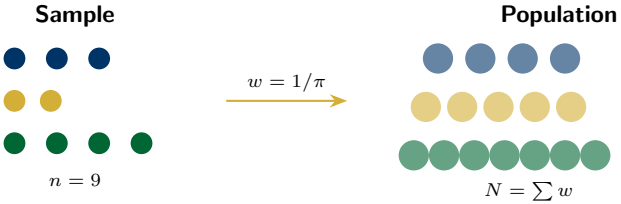
The Balloon Analogy

Each sampled household = **deflated balloon**

The **weight** = air we pump in

Fully inflated = fill space of **entire population**

Inflating Sample to Population



Different geography types ⇒ different "inflation" needed.

Why Different "Balloon" Sizes?

Urban
Oversampled
$\pi = 1/200$
$\boxed{w = 200}$

Traditional
Proportional
$\pi = 1/800$
$\boxed{w = 800}$

Farms
Undersampled
$\pi = 1/2000$
$\boxed{w = 2000}$

Smaller $\pi \Rightarrow$ larger weight

The Core Principle

Each sampled unit represents

$$\frac{1}{\pi}$$

units in the population

Where π = selection probability

The Base Weight Formula

Design Weight

$$\boxed{w_d = \frac{1}{\pi_i}}$$

Base weight = Inverse selection probability

w_d = **Design Weight**
Pop. units represented

π_i = **Inclusion Prob.**
Prob. of selection

$1/\pi$ = **Inverse**
Likelihood → count

Multi-Stage Sampling Weights

For GHS's two-stage stratified design:

$$\pi_{hij} = \pi_h \times \pi_{i|h} \times \pi_{j|hi}$$

$\pi_h = 1$
Stratum
All included

$\pi_{i|h} = \frac{a_h M_{hi}}{M_h}$
Stage 1: PSU
PPS selection

$\pi_{j|hi} = \frac{m_{hi}}{M^*_{hi}}$
Stage 2: DU
Systematic

Base weight combines all stages: $w_d = 1/\pi_{hij}$

Complete Base Weight Formula

$$w_{hij} = \underbrace{\frac{M_h}{a_h \times M_{hi}}}_{\text{Stage 1}} \times \underbrace{\frac{M^*_{hi}}{m_{hi}}}_{\text{Stage 2}}$$

M_h	Total MOS in stratum
a_h	PSUs selected
M_{hi}	MOS of PSU i
M^*_{hi}	Updated DU count
m_{hi}	DUs selected

Lindiwe's Discovery: Raw vs. Weighted

Zambara: Lindiwe calculates both estimates for urban share...

Unweighted	**Weighted**
8,547 / 16,752	6.1M / 18.2M
$= \mathbf{51.0\%}$	$= \mathbf{33.5\%}$

Caution: Unweighted overstates urban by 17.5 pp (oversampled).

Why Do We Oversample?

Reasons for disproportionate sampling:

1. **Precision**—reliable small-domain estimates
2. **Cost**—urban cheaper to enumerate
3. **Analysis**—comparing groups needs similar n
4. **Rare populations**—oversample to capture enough

Self-weighting ideal:
All weights equal: $w_i = N/n$
Minimizes variance but sacrifices precision.

Oversampling $\Rightarrow$ variable weights $\Rightarrow$ weighting essential

The Horvitz-Thompson Estimator

$$\hat{Y}_{HT} = \sum_{i \in s} \frac{y_i}{\pi_i} = \sum_{i \in s} w_i \cdot y_i$$

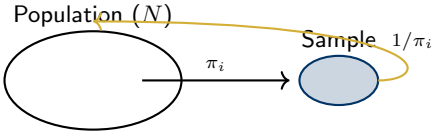
Key properties:

- **Design-unbiased**: $E[\hat{Y}_{HT}] = Y$
- **Requires known** π_i
- **No model assumptions**

When weighted by $1/\pi_i$: $E[\sum w_i] = N$

Why Does $1/\pi$ Work?

Fishing analogy:
Catch 1 fish with 1% probability. How many fish in pond?
Estimate $= 1/0.01 = 100$ fish



The weight reverses selection, projecting back to population.

Quality Check: Sum of Weights

$$\sum_{i \in s} w_i \approx N$$

Sum of weights $\approx$ population size

Zambara check:

$\sum w = 18,234,567$ Census $= 18,200,000$ Diff: $+0.19\%$ ✓

If mismatch: Check frame coverage, calculations, non-response.

GHS 2024: Base Weight Distribution

Province	n	Mean	Min	Max	Sum
Western Cape	1,931	1,137	192	3,450	2.2M
KwaZulu-Natal	3,257	1,040	285	3,890	3.4M
Gauteng	5,529	1,082	195	4,250	6.0M
(6 more...)					
Total	**20,943**	**870**	**192**	**4,250**	**18.2M**

Weight Variability Matters

Variable weights increase variance:

$$DEFF_w = 1 + CV^2(w) = 1 + \frac{Var(w)}{[\bar{w}]^2}$$

Zambara Example:
$\bar{w} = 870$, $\sigma_w = 412$
$CV = 0.47$
$DEFF_w = 1.22$

$n_{eff} = 20,943/1.22$
$= 17,167$
"Lost" 3,800 obs.

Effective Sample Size

$$n_{eff} = \frac{(\sum w_i)^2}{\sum w_i^2}$$

SRS size for same precision

Interpretation:

- $n_{eff} = n$: equal weights
- $n_{eff} < n$: variable weights
- $n_{eff} \ll n$: highly variable

Rule of thumb:

If $n_{eff}/n < 0.5$:
Consider trimming or redesign.

Lindiwe Shows the Minister

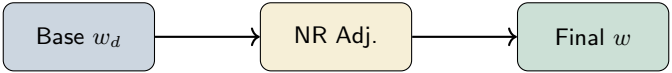
Thursday – Minister's Office

Indicator	Unwt.	Wt.	Bias
Urban share	51.0%	33.5%	+17.5 pp
Mean income	R8,450	R6,230	+R2,220
Food insecurity	18.2%	24.7%	-6.5 pp
Piped water	78.5%	62.3%	+16.2 pp

Caution: Unweighted favors urban/wealthier HHs (oversampled + higher response).

"What About Non-Response?"

Minister: "If only 16,752 from 20,940 responded, aren't base weights wrong?"
Lindiwe: "Exactly. We need NR adjustment..."



Preview: Module 3.3 covers NR adjustment

Lab 3.1: Non-Response by Geography

Lab 3.1 simulation (GHS 2024, 20% simulated NR):

GeoType	Sampled	Resp.	RR	NR Factor
Urban	13,654	9,537	69.8%	1.43
Traditional	6,565	5,404	82.3%	1.22
Farms	721	652	90.4%	1.11
Total	**20,940**	**16,752**	**80.0%**	—

Urban needs 43% inflation vs. farms 11%—differential NR.

Lab 3.1: Weight Distribution Shift

Weight Distribution

[Insert: weight_distribution_comparison.png]

Blue=Base; Green=NR-adjusted

Observations:

- NR-adjusted shift right
- More spread out
- Higher $DEFF_w$

Trade-off: reduces bias but increases variance.

Lab 3.1: Adjustment Factors by Province

[Insert: adjustment_factors_by_class.png]

Red bars (Urban) highest—more NR, more adjustment needed.

Sum of Weights: Before and After

Stage	N	$\sum w$	%
Full Sample	20,940	18.2M	100%
Respondents (Base)	16,752	14.6M	80%
Respondents (Adj.)	16,752	18.2M	100%

Magic: 16,752 respondents' adjusted weights represent all 18.2M households.

Weighting Class Method

$$w_{nr} = w_d \times \frac{n_c}{n_c^r}$$

n_c = sampled; n_c^r = responded

Lab 3.1: Province $\times$ GeoType (27 cells)

	Urban	Trad.	Farms
W. Cape	1.36	—	1.04
E. Cape	1.47	1.22	1.28
Limpopo	1.50	1.25	1.13

When Weights Go Wrong

Common pathologies:

Extreme weights
100x larger than others
Fix: weight trimming

Zero/negative
Calibration issue
Fix: bounded calibration

Sum mismatch
Check frame, calcs, coverage

Implausible estimates
Extreme weight + outlier
Fix: robust estimation

Key Takeaways

1. **Unweighted lies**—reflects sample, not population
2. **Base weight** $= 1/\pi$
3. **Balloon**: inflate each obs. to population share
4. **Multi-stage**: multiply probabilities
5. **Sum check**: $\sum w \approx N$
6. **Variability**: reduces n_{eff}

Next: Module 3.2—Base Weight Calculation (R & STATA code)

Discussion Questions

1. What drives biggest π differences in your survey?
2. Who over/under-represented with unweighted estimates?
3. Weights 50–15,000: Problem? Solution?
4. Colleague: "Weighting adds noise." Response?

5 minutes discussion

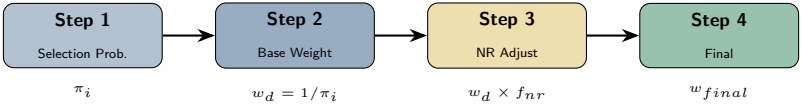
Module 3.1 – Part 2

Constructing the Weight

From Selection Probabilities to Final Weights

Process Flow with R & STATA Code

The Weighting Pipeline



This section: Steps 1–3 with R and STATA implementation

Step 1

Selection Probabilities

Computing π_i for multi-stage designs

GHS 2024: Two-Stage Stratified Design

Stage 1: PSU Selection

- 248 strata (Prov $\times$ Geo)
- PPS systematic sampling
- a_h PSUs per stratum

Stage 2: DU Selection

- Listing within PSUs
- Systematic sampling
- m_{hi} DUs per PSU

$$\pi_{hij} = \underbrace{\frac{a_h \cdot M_{hi}}{M_h}}_{\text{Stage 1}} \times \underbrace{\frac{m_{hi}}{M^*_{hi}}}_{\text{Stage 2}}$$

Data Requirements for Weight Calculation

Variable	Source	Purpose
`stratum`	Design	Stratification ID
`psu`	Design	Cluster ID
`M_h`	Census	Stratum total MOS
`M_hi`	Census	PSU MOS at selection
`a_h`	Design	PSUs selected
`M_hi_star`	Listing	Updated DU count
`m_hi`	Design	DUs selected

GHS provides `house_wgt` pre-calculated.

R: Loading GHS Data

```r
library(tidyverse)
library(survey)
library(haven)

# Load GHS 2024 household data
ghs_hh <- read_dta("GHS2024_Household.dta")

dim(ghs_hh)
# [1] 20940   190

summary(ghs_hh$house_wgt)
#    Min.  1st Qu.  Median    Mean  3rd Qu.    Max.
#   192     523      756     870     1082    4250
```

STATA: Loading GHS Data

```
use "GHS2024_Household.dta", clear

count
* 20940

summarize house_wgt, detail
*        Mean           870
*    Std. Dev.          412
*         Min           192
*         Max          4250
```

Weight Distribution by Geography

Overall:

- Range: 192 – 4,250
- Mean: 870
- CV: 0.47
- $\sum w = 18.2$M

GeoType	Mean w	n
Urban	712	13,654
Traditional	1,156	6,565
Farms	1,834	721

Farms largest weights (undersampled)

R: Examining Weights by Geography

```
ghs_hh %>%
  group_by(geotype) %>%
  summarise(
    n = n(),
    mean_wgt = mean(house_wgt),
    sum_wgt = sum(house_wgt)
  )

#   geotype          n  mean_wgt    sum_wgt
# 1 Urban        13654       712   9721048
# 2 Traditional   6565      1156   7590940
# 3 Farms          721      1834    921579

sum(ghs_hh$house_wgt)
# [1] 18233567
```

STATA: Examining Weights by Geography

```
tabstat house_wgt, by(geotype) stat(n mean sum)

* geotype |         N      mean       sum
* --------+------------------------------
*   Urban |     13654       712   9721048
*   Trad. |      6565      1156   7590940
*   Farms |       721      1834    921579
* --------+------------------------------
*   Total |     20940       870  18233567
```

Step 2

Base Weight

$$w_d = 1/\pi_i$$

Base Weight Formula

$$w_d = \frac{M_h}{a_h \cdot M_{hi}} \times \frac{M^*_{hi}}{m_{hi}}$$

Example: Western Cape Urban PSU

$M_h = 1.2\text{M}$, $a_h = 45$, $M_{hi} = 2500$, $M^*_{hi} = 2650$, $m_{hi} = 10$

$w_d = \frac{1{,}200{,}000}{45 \times 2{,}500} \times \frac{2{,}650}{10} = 10.67 \times 265 = \mathbf{2,827}$

R: Base Weight Calculation

```
# If calculating from design parameters:
# ghs_hh <- ghs_hh %>%
#   mutate(
#     pi_total = (a_h * M_hi / M_h) * (m_hi / M_hi_star),
#     base_wgt = 1 / pi_total
#   )

# GHS provides weights - use directly
ghs_hh$base_wgt <- ghs_hh$house_wgt

summary(ghs_hh$base_wgt)
#    Min.  1st Qu.  Median    Mean  3rd Qu.    Max.
#   192     523      756     870     1082    4250
```

STATA: Base Weight Calculation

```
* If calculating from design:
* gen pi_total = (a_h * M_hi / M_h) * (m_hi / M_hi_star)
* gen base_wgt = 1 / pi_total

* GHS provides weights - use directly
gen base_wgt = house_wgt

summarize base_wgt
*     Variable |    Obs      Mean    Std.Dev.     Min       Max
* -------------+---------------------------------------------
*     base_wgt | 20940       870         412      192      4250
```

Step 3

Non-Response Adjustment

Compensating for missing respondents

Why Non-Response Adjustment?

GHS Response:

- Selected: 20,940
- Responded: 16,752
- Overall RR: 80%

Differential NR:
Urban 70% — Trad 82% — Farms 90%

Without adjustment:

- $\sum w_{resp} < N$
- Urban underrepresented
- Biased estimates

Solution:
Inflate respondent weights

The Weighting Class Method

$$w_{nr} = w_d \times \frac{n_c}{n_c^r}$$

NR-adjusted = Base × Factor

- n_c = Sampled in class c
- n_c^r = Respondents in class c
- $f_{nr} = n_c / n_c^r$ = Adjustment factor

Lab 3.1: Province × GeoType = 27 weighting classes

Lab 3.1: Simulating Non-Response

Zambara: Lindiwe simulates differential NR.

GeoType	Target RR	NR Prob.
Urban	70%	0.30
Traditional	82%	0.18
Farms	90%	0.10

GHS has only respondents; simulation shows full NR process.

R: Simulating Non-Response (Lab 3.1)

```
set.seed(2024)
nr_probs <- c("Urban" = 0.30, "Traditional" = 0.18, "Farms" = 0.10)

ghs_hh <- ghs_hh %>%
  mutate(
    nr_prob = nr_probs[as.character(geotype)],
    response = rbinom(n(), 1, prob = 1 - nr_prob)
  )

ghs_hh %>% group_by(geotype) %>%
  summarise(n = n(), resp = sum(response), RR = mean(response))

#   geotype          n  resp    RR
# 1 Urban        13654  9537 0.698
# 2 Traditional   6565  5404 0.823
# 3 Farms          721   652 0.904
```

STATA: Simulating Non-Response (Lab 3.1)

```
set seed 2024

gen nr_prob = cond(geotype==1, 0.30, cond(geotype==2, 0.18, 0.10))
gen response = runiform() > nr_prob

tab geotype response, row

*   geotype |         0          1 |     Total
* ----------+----------------------+----------
*     Urban |     4,117      9,537 |    13,654
*           |     30.2%      69.8% |    100.0%
*     Trad. |     1,161      5,404 |     6,565
*           |     17.7%      82.3% |    100.0%
*     Farms |        69        652 |       721
*           |      9.6%      90.4% |    100.0%
```

Creating Weighting Classes

R:

```
ghs_hh <- ghs_hh %>%
  mutate(wgt_class =
    paste(prov, geotype, sep="_"))

n_distinct(ghs_hh$wgt_class)
# [1] 27
```

STATA:

```
egen wgt_class = group(prov geotype)

tab wgt_class
* 27 categories
```

9 provinces $\times$ 3 geotypes $=$ 27 cells

Calculating NR Factors

$$f_{nr} = \frac{n_c}{n_c^r} = \frac{1}{RR_c}$$

Example: Western Cape Urban

Sampled: 1,523 — Responded: 1,065 — RR: 69.9%

$f_{nr} = 1523/1065 = \mathbf{1.43}$

Each WC Urban respondent now represents 1.43 sampled HHs.

R: Calculating NR Factors (Lab 3.1)

```
nr_factors <- ghs_hh %>%
  group_by(wgt_class) %>%
  summarise(
    n_sam = n(),
    n_resp = sum(response),
    nr_factor = n_sam / n_resp
  )

nr_factors %>% arrange(desc(nr_factor)) %>% head(5)

#   wgt_class        n_sam n_resp nr_factor
# 1 Limpopo_Urban      567    378      1.50
# 2 EC_Urban           892    602      1.48
# 3 Gauteng_Urban    5234   3557      1.47
# 4 NC_Urban           423    291      1.45
# 5 KZN_Urban         1234    857      1.44
```

STATA: Calculating NR Factors (Lab 3.1)

```
bysort wgt_class: gen n_sam = _N
bysort wgt_class: egen n_resp = total(response)
gen nr_factor = n_sam / n_resp

tab wgt_class, sum(nr_factor)

*          wgt_class |      Mean       Freq.
* -------------------+----------------------
*      Limpopo_Urban |      1.50         567
*          EC_Urban |      1.48         892
*      Gauteng_Urban |      1.47        5234
*          NC_Urban |      1.45         423
*         KZN_Urban |      1.44        1234
```

Lab 3.1 Output: NR Adjustment Factors

Province	Urban	Traditional	Farms
Western Cape	1.43	—	1.11
Eastern Cape	1.48	1.22	1.13
Northern Cape	1.45	—	1.09
Free State	1.43	1.19	1.12
KwaZulu-Natal	1.44	1.21	1.10
North West	1.46	1.20	1.08
Gauteng	1.47	1.23	1.14
Mpumalanga	1.45	1.18	1.11
Limpopo	1.50	1.25	1.13

Pattern: Urban 43–50% inflation needed; Farms only 8–14%.

R: Applying NR Adjustment (Lab 3.1)

```
ghs_hh <- ghs_hh %>%
  left_join(nr_factors %>% select(wgt_class, nr_factor),
            by = "wgt_class") %>%
  mutate(wgt_nr = ifelse(response == 1, base_wgt * nr_factor, NA))

# Compare weight sums
ghs_hh %>% filter(response == 1) %>%
  summarise(n = n(), sum_base = sum(base_wgt), sum_nr = sum(wgt_nr))

#        n   sum_base      sum_nr
# 1 16752  14587654   18233567
```

Success! Sum restored: 14.6M → 18.2M

STATA: Applying NR Adjustment (Lab 3.1)

```
gen wgt_nr = base_wgt * nr_factor if response == 1

* Compare sums
total base_wgt if response == 1
*    Total: 14,587,654

total wgt_nr if response == 1
*    Total: 18,233,567

display "Restored to population: 18.2M"
```

Result: 16,752 respondents now represent all 18.2M HHs.

Lab 3.1 Output: Weight Diagnostics

Metric	Base	NR-Adj	Status
Sum	14.6M	18.2M	✓
Mean	870	1,088	—
CV	0.47	0.52	✓
Min	192	211	✓
Max	4,250	6,375	✓
Max/Min	22	30	✓
$DEFF_w$	1.22	1.27	✓
n_{eff}	13,731	13,191	—

CV increased slightly. Lost ∼540 effective obs to NR adjustment.

The Bias-Variance Trade-off

Before NR Adjustment:

- Lower variance
- $n_{eff} = 13,731$
- Biased (Urban under-rep)

After NR Adjustment:

- Higher variance
- $n_{eff} = 13,191$
- Less biased

NR adjustment: $\downarrow$ bias at cost of $\uparrow$ variance
Usually worthwhile when NR is differential.

Module 3.1 Part 2: Key Takeaways

1. **Step 1:** Selection probability π_i from design
2. **Step 2:** Base weight $w_d = 1/\pi_i$
3. **Step 3:** NR adjustment $w_{nr} = w_d \times (n_c/n_c^r)$
4. **Weighting classes** homogeneous for response propensity
5. **Verify:** $\sum w_{nr} \approx N$
6. **Trade-off:** Less bias, more variance

Questions?

Next: Module 3.2 – Calibration Methods

Day 3 – Module 3.2

Introduction to Calibration

Aligning Survey Estimates to Known Totals

Post-Stratification • Raking • GREG

Module 3.2 Overview

Learning Objectives:

1. Explain why calibration improves survey estimates
2. Compare post-stratification, raking, and GREG methods
3. Interpret the Deville & Särndal (1992) framework
4. Implement calibration in R and STATA

Duration: 75 minutes — **Lab:** 3.2 – Raking Implementation

Zambara: Lindiwe calibrates to census population totals.

The Problem: Why Calibrate?

Zambara: After NR adjustment, Lindiwe checks her totals...

Survey Estimates:

Province	Sum of w_{nr}
Gauteng	5,823,412
KZN	3,456,789
Western Cape	2,012,345
...	...
Total	**19,609,606**

Census/Admin Data:

Province	N_{pop}
Gauteng	5,400,000
KZN	3,200,000
Western Cape	2,150,000
...	...
Total	**18,390,000**

Problem: Survey overestimates population by 1.2 million (6.6%)!

Calibration: The Solution

Calibration adjusts weights so that weighted survey totals match known population totals ("benchmarks" or "controls").

Key idea:

- We *know* population totals from census/admin data
- Survey estimates should *reproduce* these known totals
- Adjust weights minimally to achieve this alignment

Result:

- Reduced bias for variables correlated with calibration variables
- Often reduced variance (borrowed strength from auxiliary data)
- Internal consistency with official statistics

Three Calibration Methods

Post-Stratification	**Raking**	**GREG**
Exact cell counts	Marginal totals	Regression approach
Simple	Iterative	Flexible
Cross-classification	Multiple margins	Continuous vars

Increasing flexibility

All three are special cases of calibration estimators
(Deville & Särndal, 1992)

Method 1

Post-Stratification

Calibrating to cell totals

Post-Stratification: The Idea

Divide population into cells, adjust each cell to match known total.

	Male	Female	Male	Female
Urban	2,450	2,680	1,230	1,340
Rural				
Farm				

Survey sum w

Adjust →

2,500	2,700	1,200	1,300	

Census N

$$w_{ps,i} = w_{nr,i} \times \frac{N_c}{\sum_{j \in c} w_{nr,j}}$$

Post-Stratification: Requirements

Requires:

- Known population totals for *every cell*
- Sufficient sample in each cell ($n_c \geq 20$ rule of thumb)

Example: Province $\times$ Sex = 18 cells

	Male	Female	Total
Western Cape	1,050,000	1,100,000	2,150,000
Eastern Cape	920,000	1,030,000	1,950,000
...	...	...	...

Limitation: Need full cross-tabulation. Gets unwieldy with many variables!

Method 2

Raking

Calibrating to marginal totals

Raking: The Idea

Match marginal totals only—don't need full cross-tabulation.

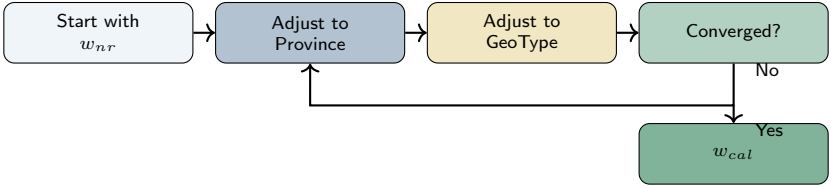
	Male	Female	
Urban	?	?	5.4M
Trad.	?	?	3.2M
Farm	?	?	2.1M
	8.9M	9.5M	

Known margins

Iterative Proportional Fitting (IPF):

1. Adjust weights to match Province totals
2. Adjust weights to match GeoType totals
3. Repeat until convergence

Raking: Iterative Process



Convergence: When adjustments become negligibly small
Typically 5–20 iterations; guaranteed to converge for consistent margins

Raking: Advantages and Limitations

Advantages:

- Only need marginal totals
- Handles many variables
- Widely implemented
- Usually converges quickly

Limitations:

- Margins must be consistent
- Can produce extreme weights
- Ignores cell interactions
- May not converge if margins inconsistent

Lab 3.2: Raking on Province (9) $\times$ GeoType (3) margins

Method 3

GREG Calibration

Generalized Regression Estimator

GREG: The Regression Approach

Use auxiliary variables to predict y, then calibrate.

$$\hat{Y}_{GREG} = \hat{Y}_{HT} + (\mathbf{X} - \hat{\mathbf{X}})'\hat{\boldsymbol{\beta}}$$

Where:

- $\hat{Y}_{HT}$ = Horvitz-Thompson estimate
- $\mathbf{X}$ = Known population totals of auxiliaries
- $\hat{\mathbf{X}}$ = Survey estimates of auxiliary totals
- $\hat{\boldsymbol{\beta}}$ = Regression coefficients

Key insight: If survey over/underestimates X, adjust Y proportionally.

GREG: Advantages and Limitations

Advantages:

- Handles continuous variables
- Single-step (no iteration)
- Optimal under model
- Natural variance estimator

Limitations:

- Can produce negative weights
- Requires more auxiliary info
- Model specification matters
- More complex to implement

Use when:

- Auxiliary variables are continuous (age, income totals)
- Strong correlation between y and auxiliaries
- Post-strat/raking produce extreme weights

Unified Framework

Deville & Särndal

(1992)

The General Calibration Estimator

Deville & Särndal: The Calibration Problem

Find new weights w_i that:

1. **Match known totals:** $\sum_s w_i \mathbf{x}_i = \mathbf{X}$
2. **Stay close to design weights:** Minimize distance from d_i

$$\text{Minimize } \sum_s G(w_i, d_i)$$
$$\text{subject to } \sum_s w_i \mathbf{x}_i = \mathbf{X}$$

$G(\cdot)$ = distance function measuring departure from design weights

Distance Functions Determine Method

Distance $G(w, d)$	**Method**	**Weights**
$(w-d)^2/2d$	Linear (GREG)	Can be negative
$w\log(w/d) - w + d$	Raking ratio	Always positive
$-d\log(w/d) + w - d$	Logit	Bounded range
$(w-d)^2/2$	Chi-squared	Can be negative

Key insight from D&S:
All calibration methods are solving the *same optimization problem* with different distance functions!

The G-Weight: Calibration Factor

$$g_i = \frac{w_{cal,i}}{w_{design,i}}$$

Calibration adjustment factor

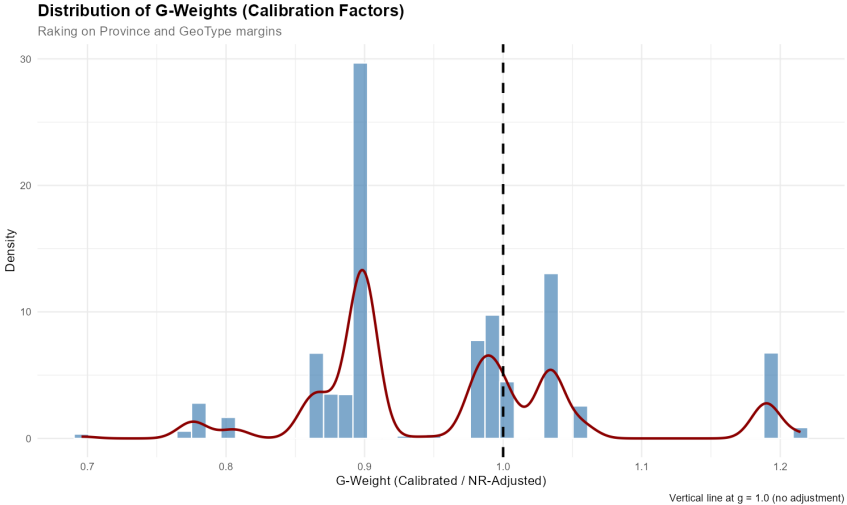
Interpretation:

- $g_i = 1.0$: No adjustment needed
- $g_i > 1.0$: Unit underrepresented, weight increased
- $g_i < 1.0$: Unit overrepresented, weight decreased

Diagnostics:

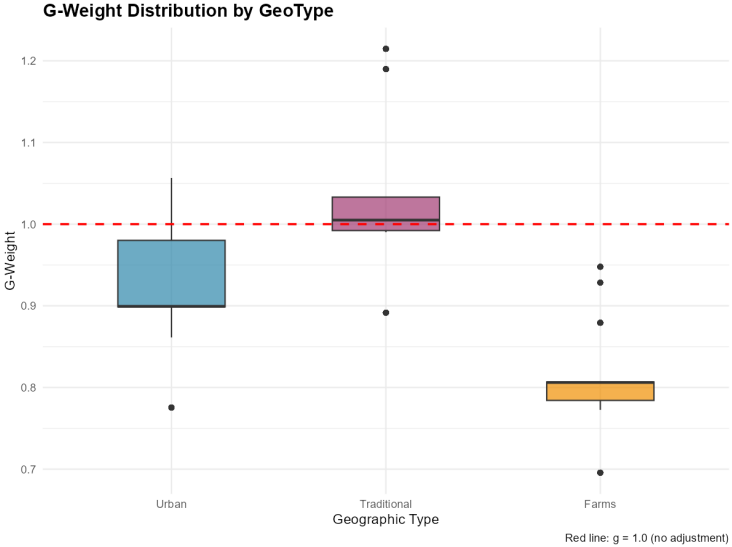
- Mean $g \approx 1.0$ (by design)
- Range should be moderate (e.g., 0.5 – 2.0)
- Extreme g values signal problems

Lab 3.2: G-Weight Distribution

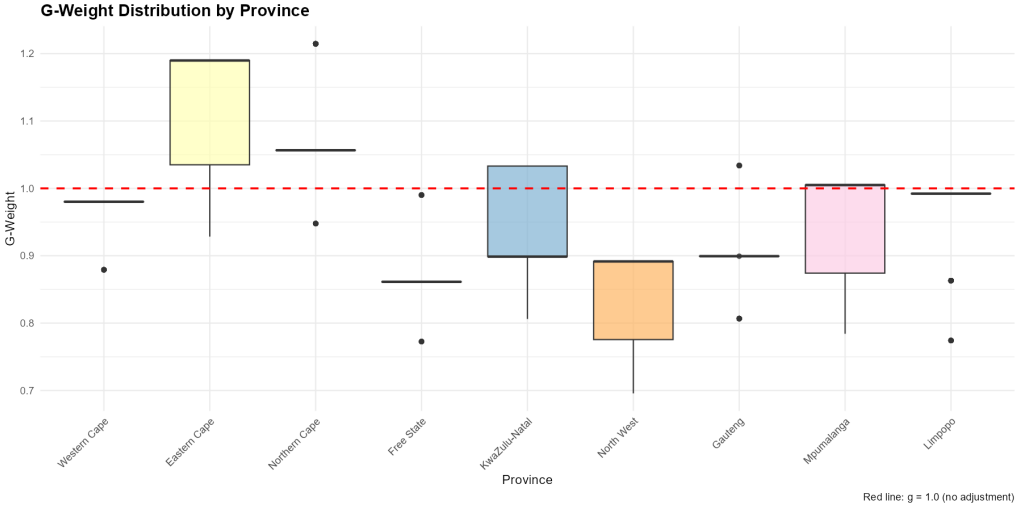


Multi-modal: reflects Province $\times$ GeoType adjustment patterns

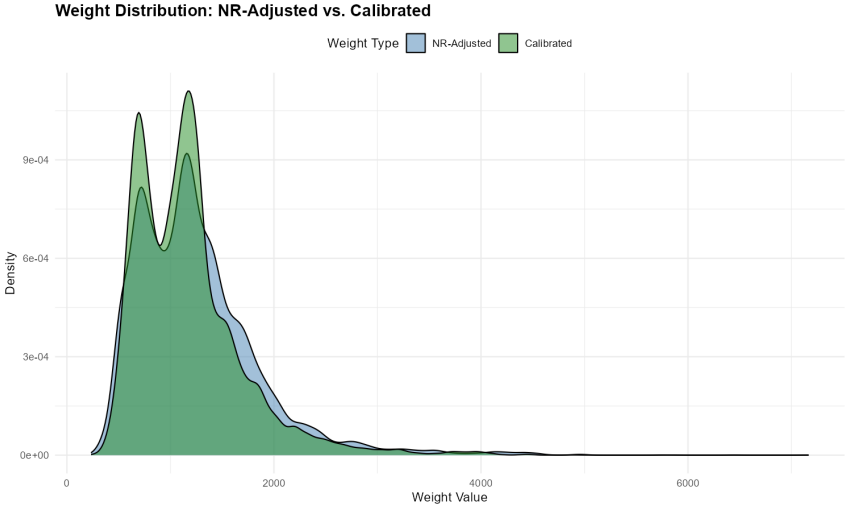
Lab 3.2: G-Weights by GeoType



Lab 3.2: G-Weights by Province



Before & After: Weight Distribution



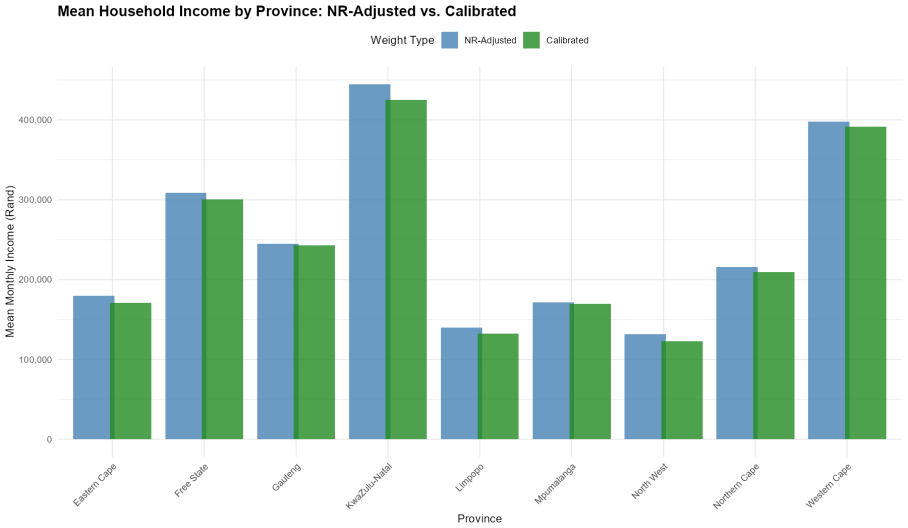
Calibrated weights: tighter distribution, lower CV

Before & After: Weight Diagnostics

Metric	NR-Adjusted	Calibrated	Change
Sum	19,609,606	18,390,000	↓ 6.2%
Mean	1,251	1,173	↓ 6.2%
Std Dev	601	530	↓ 11.8%
CV	0.48	0.45	↓ 6.3%
Min	271	233	—
Max	7,165	6,444	—
$DEFF_w$	1.23	1.20	↓ 2.4%

Result: Calibration reduced weight variability and DEFF!

Before & After: Income Estimates by Province



Lab 3.2: Income Estimate Changes

Province	NR Mean	Cal Mean	Diff	Pct
Western Cape	R 397,878	R 391,674	-6,204	-1.6%
Eastern Cape	R 179,630	R 171,033	-8,597	-4.8%
Northern Cape	R 215,474	R 209,404	-6,070	-2.8%
Free State	R 308,557	R 300,353	-8,204	-2.7%
KwaZulu-Natal	R 444,707	R 425,121	-19,586	-4.4%
North West	R 131,872	R 123,107	-8,765	-6.7%
Gauteng	R 245,015	R 242,863	-2,152	-0.9%
Mpumalanga	R 171,808	R 169,893	-1,915	-1.1%
Limpopo	R 140,029	R 132,396	-7,633	-5.5%

All estimates decreased after calibration (was overestimating)

Choosing a Calibration Method

Method	Use When	Auxiliary Data
Post-Strat	Few variables, large cells	Full cross-tabulation
Raking	Multiple categorical vars	Marginal totals only
GREG	Continuous auxiliaries	Means/totals of cont. vars

Decision tree:

1. Do you have full cross-tabs? $\rightarrow$ Post-stratification
2. Only marginal totals? $\rightarrow$ Raking
3. Continuous auxiliaries? $\rightarrow$ GREG
4. Extreme weights? $\rightarrow$ Bounded calibration

Common Calibration Problems

Problem 1: No convergence
Margins inconsistent
Solution: Check totals sum correctly

Problem 2: Extreme weights
$g > 3$ or $g < 0.3$
Solution: Weight trimming or bounded calibration

Problem 3: Negative weights
Linear calibration issue
Solution: Use raking or logit distance

Problem 4: Small cells
Unstable adjustments
Solution: Collapse cells or use margins only

Lindiwe's Calibration Success

Zambara NSO: Lindiwe presents calibrated estimates to the Minister.

What calibration achieved:

- Survey totals now match census: 18.39M households ✓
- Province distributions aligned to admin data ✓
- GeoType proportions corrected ✓
- Weight variability reduced (CV: 0.48 → 0.45) ✓

Minister's response:
"Now I can cite these statistics with confidence—they're consistent with our census!"

Module 3.2: Key Takeaways

1. **Calibration** aligns survey estimates to known population totals
2. **Post-stratification**: Exact cell totals (simple, requires full cross-tab)
3. **Raking**: Marginal totals only (flexible, iterative)
4. **GREG**: Regression approach (handles continuous vars)
5. **Deville & Särndal**: All methods minimize distance from design weights
6. **G-weights**: Monitor calibration adjustment factors
7. **Benefits**: Reduced bias, often reduced variance, consistency

Questions?

Next: Module 3.3 – Variance Estimation

Day 3 – Module 3.3

Implementing Calibration

Lab 3.2: Raking to Census Totals

R `survey::rake()` • STATA `ipfraking`

Module 3.3 Overview

Learning Objectives:

1. Define population control totals from census data
2. Implement raking calibration in R and STATA
3. Calculate and interpret g-weights
4. Verify calibration success and diagnose problems

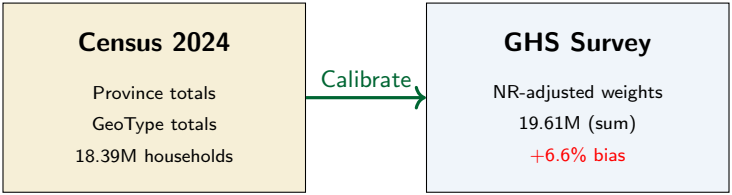
Duration: 60 minutes (hands-on lab)

Prerequisites:

- NR-adjusted weights from Lab 3.1
- Census population totals by Province and GeoType

Lindiwe's Golden Standard

Zambara NSO: The Census Bureau delivers population totals...



"The Census is our golden standard. Survey estimates must align."
— Director of Methodology, ZNSO

Step 1: Define Population Control Totals

Two sets of marginal totals for raking:

Province (9 categories):

Province	N_{pop}
Western Cape	2,150,000
Eastern Cape	1,950,000
Northern Cape	420,000
Free State	890,000
KwaZulu-Natal	3,200,000
North West	1,180,000
Gauteng	5,400,000
Mpumalanga	1,450,000
Limpopo	1,750,000
Total	**18,390,000**

GeoType (3 categories):

GeoType	N_{pop}
Urban	12,500,000
Traditional	5,200,000
Farms	690,000
Total	**18,390,000**

Critical: Both margins must sum to same total!

R: Define Population Totals

```r
# Province population totals (from Census 2024)
pop_province <- c(
  "Western Cape"  = 2150000,
  "Eastern Cape"  = 1950000,
  "Northern Cape" =  420000,
  "Free State"    =  890000,
  "KwaZulu-Natal" = 3200000,
  "North West"    = 1180000,
  "Gauteng"       = 5400000,
  "Mpumalanga"    = 1450000,
  "Limpopo"       = 1750000
)

# GeoType population totals
pop_geotype <- c(
  "Urban"       = 12500000,
  "Traditional" =  5200000,
  "Farms"       =   690000
)

# Verify totals match
sum(pop_province)  # 18390000
sum(pop_geotype)   # 18390000
```

STATA: Define Population Totals

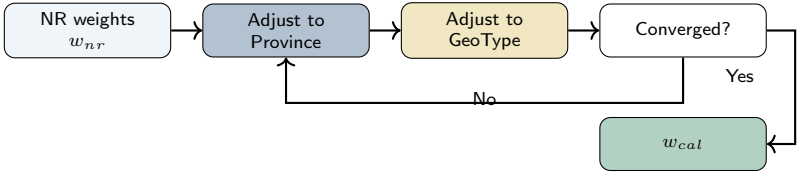
```
* Create population totals dataset
clear
input str20 prov long pop_total
"Western Cape"   2150000
"Eastern Cape"   1950000
"Northern Cape"   420000
"Free State"      890000
"KwaZulu-Natal"  3200000
"North West"     1180000
"Gauteng"        5400000
"Mpumalanga"     1450000
"Limpopo"        1750000
end
save pop_province.dta, replace

* GeoType totals
clear
input byte geotype long pop_total
1 12500000
2  5200000
3   690000
end
save pop_geotype.dta, replace
```

Pre-Calibration Check: Survey vs Census

Province	**Survey** $\sum w_{nr}$	**Census** N	**Ratio**
Western Cape	2,287,431	2,150,000	1.064
Eastern Cape	2,078,654	1,950,000	1.066
Northern Cape	447,892	420,000	1.066
Free State	948,123	890,000	1.065
KwaZulu-Natal	3,412,567	3,200,000	1.066
North West	1,258,234	1,180,000	1.066
Gauteng	5,756,789	5,400,000	1.066
Mpumalanga	1,545,678	1,450,000	1.066
Limpopo	1,874,238	1,750,000	1.071
Total	**19,609,606**	**18,390,000**	**1.066**

Survey overestimates by $\sim$6.6% across all provinces

Step 2: Implement Raking



Convergence criterion: Maximum change $< 10^{-6}$ between iterations

Typical iterations: 5–15 for well-behaved data

R: The rake() Function

```r
library(survey)

# Create survey design with NR-adjusted weights
des_nr <- svydesign(
  id = ~psu, strata = ~stratum,
  weights = ~wgt_nr, data = ghs_resp
)

# Define margins for raking
pop_margins <- list(
  ~prov,      # First margin: Province
  ~geotype    # Second margin: GeoType
)

pop_totals <- list(
  pop_province,  # Named vector of province totals
  pop_geotype    # Named vector of geotype totals
)

# Perform raking
des_cal <- rake(des_nr,
                sample.margins = pop_margins,
                population.margins = pop_totals,
                control = list(maxit = 50, epsilon = 1e-6))
```

STATA: The ipfraking Command

```
* Install if needed: ssc install ipfraking

* Merge population totals
merge m:1 prov using pop_province.dta, nogen
rename pop_total prov_target
merge m:1 geotype using pop_geotype.dta, nogen
rename pop_total geo_target

* Perform raking
ipfraking [pw=wgt_nr], ///
    generate(wgt_cal) ///
    ctotal(prov prov_target) ///
    ctotal(geotype geo_target) ///
    iterate(50) tolerance(1e-6)

* Check convergence
display "Converged in " r(iterations) " iterations"
```

Alternative: `survwgt rake` or manual IPF loop

Checking Convergence

R Output:

```
# Check iterations
des_cal$call
# rake converged in 8
# iterations

# Verify margins
svytotal(~prov, des_cal)
#                 total      SE
# Western Cape 2150000   0
# Eastern Cape 1950000   0
# ...
```

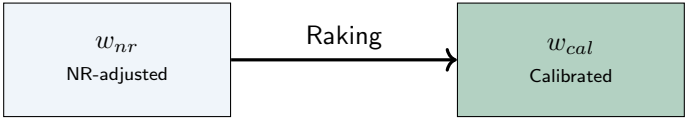
STATA Output:

```
. ipfraking ...
Iteration 1: max diff = 0.0634
Iteration 2: max diff = 0.0089
Iteration 3: max diff = 0.0012
...
Iteration 8: max diff = 0.0000
Converged.

. tab prov [aw=wgt_cal]
* Totals match targets
```

Success: Converged in 8 iterations. All margins match targets exactly.

Step 3: Extract Calibrated Weights

After raking, extract the new weights:



Sum: 19.61M
Mean: 1,251

Sum: 18.39M
Mean: 1,173

The g-weight captures the adjustment:

$$g_i = \frac{w_{cal,i}}{w_{nr,i}}$$

R: Extract Calibrated Weights

```r
# Extract calibrated weights from design object
ghs_resp$wgt_cal <- weights(des_cal)

# Calculate g-weights
ghs_resp$g_weight <- ghs_resp$wgt_cal / ghs_resp$wgt_nr

# Summary statistics
summary(ghs_resp$wgt_cal)
#    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#     233     480     720    1173    1650    6444

summary(ghs_resp$g_weight)
#    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#   0.783   0.891   0.937   0.938   0.982   1.213

# Verify sum matches target
sum(ghs_resp$wgt_cal)
# [1] 18390000
```

STATA: Extract Calibrated Weights

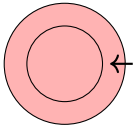
```
* wgt_cal already created by ipfraking

* Calculate g-weights
gen g_weight = wgt_cal / wgt_nr

* Summary statistics
summarize wgt_cal, detail
*        Percentiles      Smallest
*  1%          267            233
*  5%          312            241
* 10%          378            256
* ...
* Mean         1173
* Sum      18390000

summarize g_weight, detail
* Mean      0.938
* Min       0.783
* Max       1.213
```

Step 4: Understanding G-Weights

$$g_i = \frac{w_{cal,i}}{w_{nr,i}}$$

Calibration adjustment factor for unit i

$g < 1$

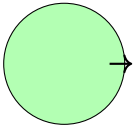


Overrepresented
Weight decreased

$g = 1$

Correctly
represented

$g > 1$



Underrepresented
Weight increased

Acceptable range: $0.3 < g < 3.0$ (rule of thumb)

R: G-Weight Diagnostics

```
# G-weight summary by GeoType
ghs_resp %>%
  group_by(geotype) %>%
  summarise(
    n = n(),
    mean_g = mean(g_weight),
    min_g = min(g_weight),
    max_g = max(g_weight),
    pct_extreme = mean(g_weight < 0.5 | g_weight > 2) * 100
  )

#   geotype          n mean_g min_g max_g pct_extreme
# 1 Urban         9537  0.912 0.783 1.087        0.0
# 2 Traditional   5404  0.987 0.856 1.213        0.0
# 3 Farms          652  0.923 0.812 1.098        0.0

# No extreme g-weights - good calibration!
```

STATA: G-Weight Diagnostics

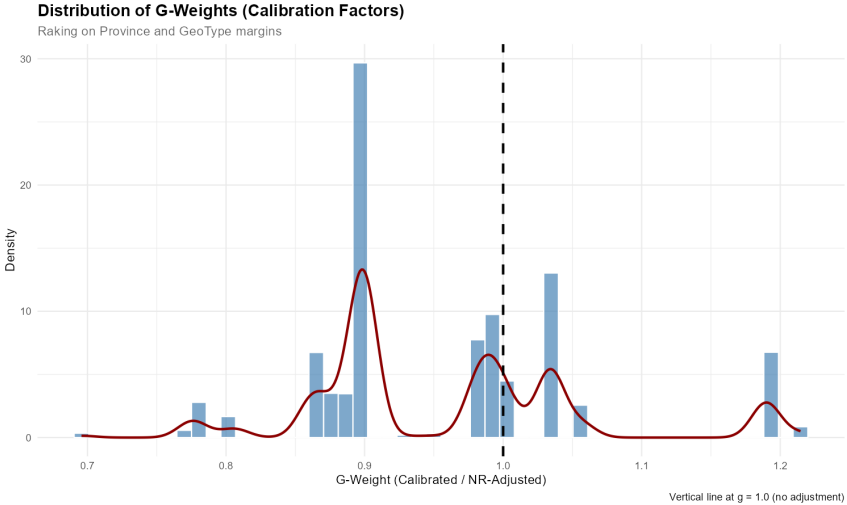
```
* G-weight summary by GeoType
tabstat g_weight, by(geotype) stat(n mean min max)

* geotype |         N      mean       min       max
* --------+------------------------------------------
*   Urban |      9537     0.912     0.783     1.087
*   Trad. |      5404     0.987     0.856     1.213
*   Farms |       652     0.923     0.812     1.098
* --------+------------------------------------------
*   Total |     15593     0.938     0.783     1.213

* Check for extreme values
count if g_weight < 0.5 | g_weight > 2
* 0

* Flag extreme g-weights (if any)
gen g_extreme = (g_weight < 0.5 | g_weight > 2)
tab g_extreme
```

Interpreting G-Weight Patterns

Pattern	Diagnosis	Action
Mean $g \approx 0.94$	Survey was overestimating	Expected: NR inflated weights
g range: 0.78–1.21	Moderate adjustments	Acceptable range
Urban: $g < 1$	Urban overrepresented	Weights reduced
Traditional: $g \approx 1$	Well-represented	Minimal adjustment
No extreme g	Good calibration	No trimming needed

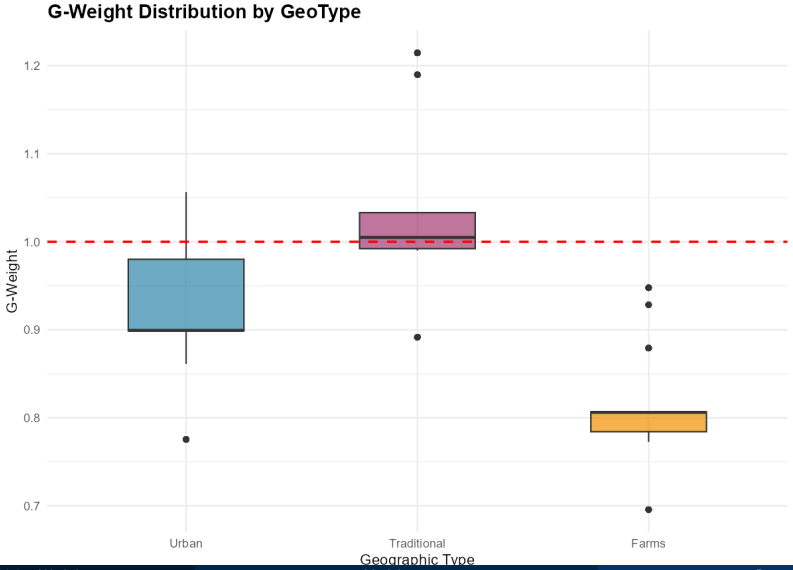
Zambara result: All g-weights within 0.78–1.21. No extreme values requiring trimming.

G-Weight Distribution

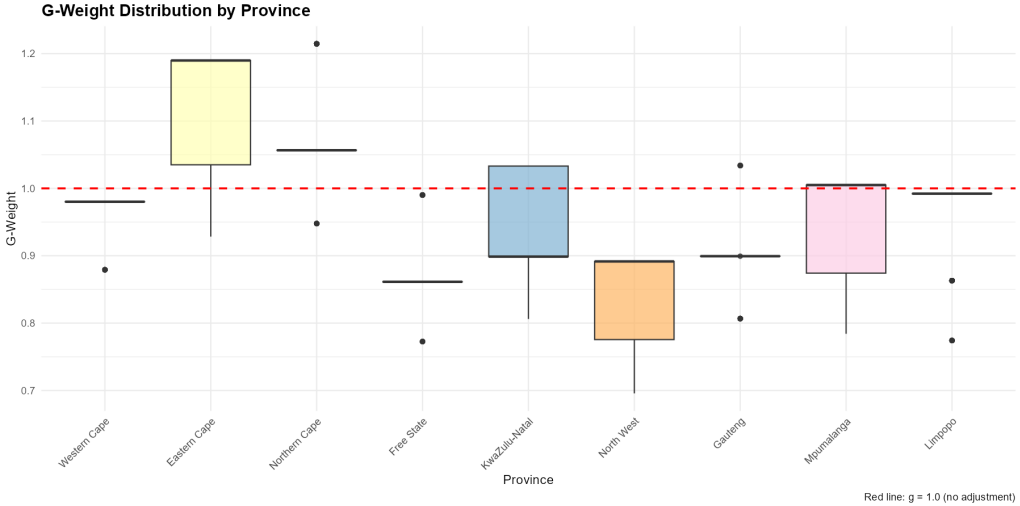


Mean $g = 0.94$ (overall downward adjustment to match census)

G-Weights by Geography Type



G-Weights by Province



R: Creating G-Weight Visualizations

```r
library(ggplot2)

# Histogram
ggplot(ghs_resp, aes(x = g_weight)) +
  geom_histogram(bins = 30, fill = "steelblue",
                 color = "white", alpha = 0.7) +
  geom_vline(xintercept = 1, linetype = "dashed",
             color = "red", size = 1) +
  labs(x = "G-Weight", y = "Frequency",
       title = "Distribution of Calibration Factors") +
  theme_minimal()

# Boxplot by GeoType
ggplot(ghs_resp, aes(x = geotype_label, y = g_weight,
                     fill = geotype_label)) +
  geom_boxplot() +
  geom_hline(yintercept = 1, linetype = "dashed") +
  labs(x = "Geography Type", y = "G-Weight") +
  theme_minimal() + theme(legend.position = "none")
```

Step 5: Verify Calibration Success

Three verification checks:

1. **Margin check:** Do weighted totals match population?
   - $\sum_s w_{cal,i} \cdot \mathbf{1}_{prov=j} = N_j$ for all provinces
   - $\sum_s w_{cal,i} \cdot \mathbf{1}_{geo=k} = N_k$ for all geotypes
2. **Total check:** Does overall sum match?
   - $\sum_s w_{cal,i} = 18,390,000$
3. **G-weight check:** Are adjustments reasonable?
   - Mean $g \approx 1$, range within bounds

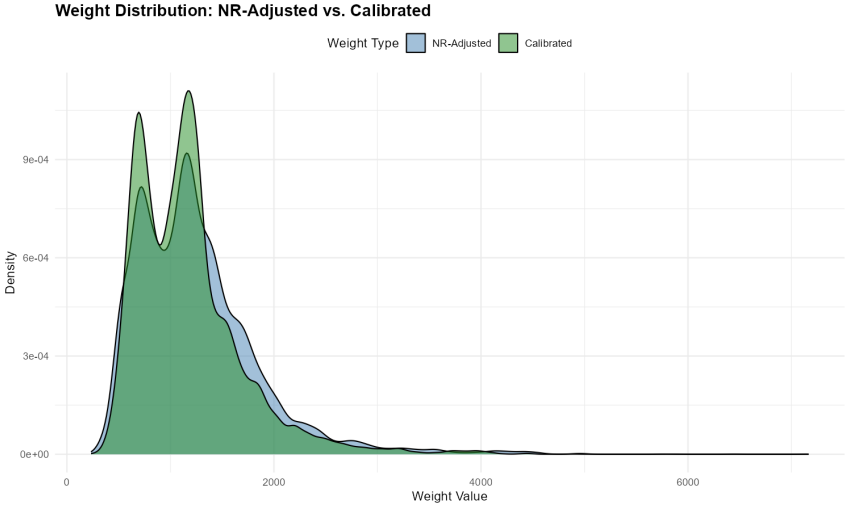
R: Verification Code

```r
# Check 1: Province margins
prov_check <- ghs_resp %>%
  group_by(prov) %>%
  summarise(survey = sum(wgt_cal)) %>%
  left_join(tibble(prov = names(pop_province),
                   census = pop_province))

prov_check %>% mutate(diff = survey - census)
# All differences = 0 (or tiny floating point)

# Check 2: GeoType margins
geo_check <- ghs_resp %>%
  group_by(geotype) %>%
  summarise(survey = sum(wgt_cal))
# Matches pop_geotype exactly

# Check 3: Total sum
sum(ghs_resp$wgt_cal)
# [1] 18390000
```

STATA: Verification Code

```
* Check 1: Province margins
tab prov [aw=wgt_cal], matcell(survey_prov)
* Compare to pop_province.dta - should match exactly

* Check 2: GeoType margins
tab geotype [aw=wgt_cal], matcell(survey_geo)
* Compare to pop_geotype.dta - should match exactly

* Check 3: Total sum
total wgt_cal
*    Total
*  --------
*  18390000

* Check 4: Cross-tabulation (not calibrated, just check)
tab prov geotype [aw=wgt_cal]
```

Before & After: Weight Comparison



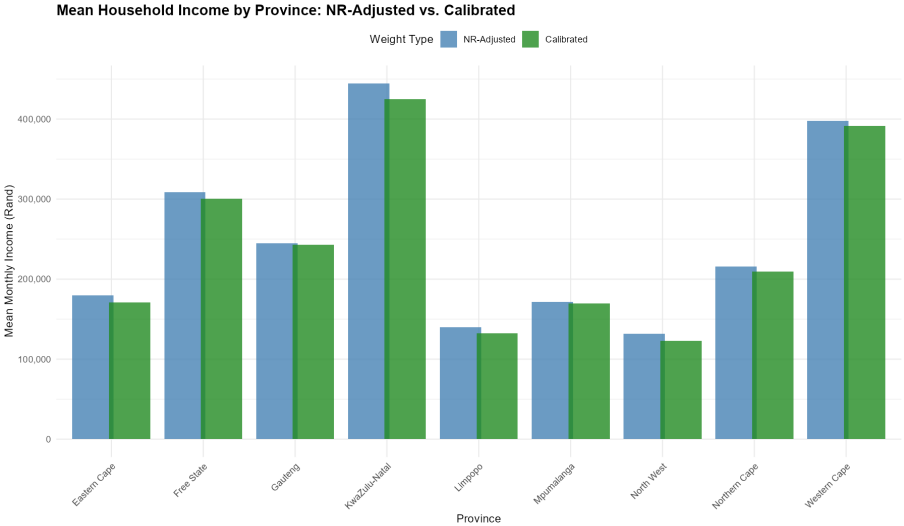
Calibrated weights: shifted left (reduced), slightly tighter distribution

Before & After: Weight Diagnostics

Metric	NR-Adjusted	Calibrated	Change
Sum	19,609,606	18,390,000	-6.2%
Mean	1,251	1,173	-6.2%
Std Dev	601	530	-11.8%
CV	0.480	0.452	-5.8%
Min	271	233	—
Max	7,165	6,444	—
$DEFF_w$	1.23	1.20	-2.4%

Bonus: Calibration reduced weight variability, improving efficiency!

Before & After: Income Estimates



All provinces: estimates decreased after calibration (correcting overestimation)

Step 6: Export Calibrated Data

R:

```r
# Save calibrated dataset
ghs_cal <- ghs_resp %>%
  select(hhid, psu, stratum,
         prov, geotype,
         wgt_nr, wgt_cal,
         g_weight)

# Multiple formats
write_csv(ghs_cal,
  "ghs2024_calibrated.csv")
write_dta(ghs_cal,
  "ghs2024_calibrated.dta")
saveRDS(des_cal,
  "ghs2024_design_cal.rds")
```

STATA:

```stata
* Keep essential variables
keep hhid psu stratum ///
     prov geotype ///
     wgt_nr wgt_cal ///
     g_weight

* Save calibrated dataset
save ghs2024_calibrated.dta, ///
     replace

* Export to CSV
export delimited using ///
  "ghs2024_calibrated.csv", ///
  replace
```

Always retain: Original weight, calibrated weight, g-weight for diagnostics

Module 3.3: Key Takeaways

1. **Define totals carefully:** Both margins must sum to same total
2. **R:** `survey::rake()` with `sample.margins` and `population.margins`
3. **STATA:** `ipfraking` with `ctotal()` for each margin
4. **Check convergence:** Should complete in < 20 iterations
5. **Calculate g-weights:** $g = w_{cal}/w_{nr}$ to diagnose adjustments
6. **Verify margins:** Weighted totals must match targets exactly
7. **Acceptable g-range:** 0.3–3.0; flag and investigate extremes

Lindiwe's survey now aligns with the Golden Standard!

Next: Module 3.4 – Variance Estimation Methods

Day 3 – Module 3.3

The Mystery of Variance

Why s^2/n Fails in Complex Surveys

Clustering • Stratification • Design Effects

The Statistician's Puzzle

Zambara Case

Lindiwe calculates the standard error for mean household income...

Naive approach:

$$SE_{naive} = \frac{s}{\sqrt{n}} = \frac{478,234}{\sqrt{20,940}} = 3,305$$

Correct approach:

$$SE_{design} = 13,642$$

The naive SE underestimates by 76%!
Why is the true uncertainty 4× larger?

The Fundamental Assumption That Fails

Simple Random Sampling Assumes:

Each observation is **independent**

Reality in household surveys:

- Households in the same PSU share characteristics
- Neighbors have similar incomes, education, housing
- Observations within clusters are **correlated**

SRS (Independent)

Clustered (Correlated)

The Naive Variance Formula

For Simple Random Sampling:

$$\boxed{Var(\bar{y}) = \frac{s^2}{n} \cdot \left(1 - \frac{n}{N}\right)}$$

Where:

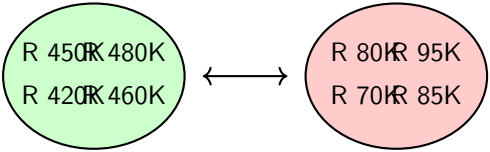
- $s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})^2$ (sample variance)
- $n =$ sample size
- $(1 - n/N) =$ finite population correction (often ≈ 1)

Key assumption: All n observations contribute equally to reducing uncertainty. Each new observation adds $1/n$ worth of "information."

Why Clustering Inflates Variance

The problem: Correlated observations provide *redundant* information.

Cluster A: Rich area **Cluster B: Poor area**



Low within-cluster variance *High between-cluster variance* *Low within-cluster variance*

8 households but only **2 independent pieces** of information about population mean!

Intra-Class Correlation (ρ)

Intra-Class Correlation

$$\rho = \frac{\sigma^2_{between}}{\sigma^2_{between} + \sigma^2_{within}}$$

Interpretation:

- $\rho = 0$: No clustering effect (observations independent)
- $\rho = 1$: Perfect clustering (all units in cluster identical)
- Typical values: $\rho = 0.02$ to 0.10 for household surveys

Variable	Typical ρ	Implication
Income	0.05–0.15	Moderate clustering
Education	0.03–0.08	Lower clustering
Housing type	0.15–0.30	High clustering
Ethnicity	0.30–0.60	Very high clustering

The Design Effect (DEFF)

Design Effect

$$DEFF = \frac{Var_{complex}(\bar{y})}{Var_{SRS}(\bar{y})}$$

For clustered designs:

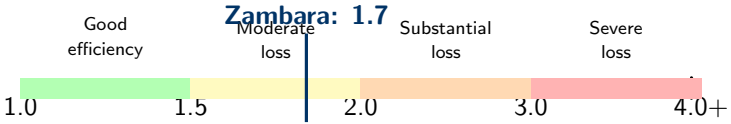
$$\boxed{DEFF \approx 1 + \rho(m-1)}$$

Where m = average cluster size

Example (Zambara GHS):

- Average cluster size: $m = 20,940/3,218 \approx 6.5$
- If $\rho = 0.05$: $DEFF = 1 + 0.05(6.5 - 1) = 1.28$
- If $\rho = 0.10$: $DEFF = 1 + 0.10(6.5 - 1) = 1.55$

Interpreting the Design Effect

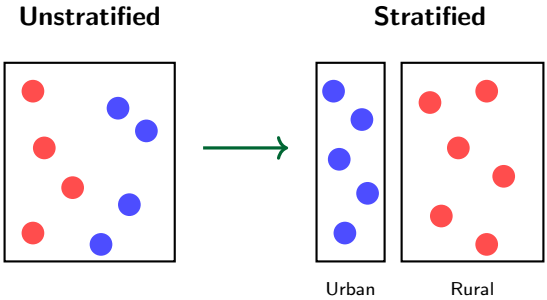


What DEFF = 1.7 means:

- Variance is 70% higher than under SRS
- Need 70% larger sample for same precision as SRS
- **Effective sample size:** $n_{eff} = n/DEFF = 20,940/1.7 = 12,318$

Stratification: The Good News

While clustering inflates variance, stratification reduces it!

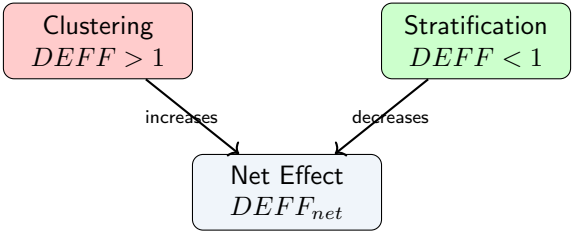


Why stratification helps:

- Removes between-stratum variance from the SE
- Guarantees representation from each stratum
- $DEFF_{strat} < 1$ when strata are homogeneous within

The Net Design Effect

Complex surveys combine both effects:



Zambara GHS example:

- Clustering effect: $\times 2.1$ (inflates variance)
- Stratification effect: $\times 0.8$ (reduces variance)
- Net DEFF: $2.1 \times 0.8 = 1.7$

Stratification partially compensates for clustering, but net effect still inflates variance.

The Correct Variance Formula

For stratified multi-stage cluster sampling:

$$\boxed{Var(\hat{\bar{Y}}) = \sum_{h=1}^{H} \frac{n_h}{n_h - 1} \sum_{i=1}^{n_h} (z_{hi} - \bar{z}_h)^2}$$

Where:

- H = number of strata
- n_h = number of PSUs in stratum h
- $z_{hi} = \sum_j w_{hij} y_{hij}$ (weighted cluster total)
- $\bar{z}_h$ = mean of cluster totals in stratum h

Key insight: Variance is calculated *between PSUs within strata*, not between individual observations!

The Ultimate Cluster Principle

Key Insight

With probability sampling at the first stage, variance estimation only requires variation **between PSUs**, regardless of subsequent stages.

Practical implication:

PSU Level: z_1 z_2 z_3 $\longrightarrow$ $Var(\hat{Y})$

- Only need PSU-level summaries (z_{hi})
- Within-PSU selection details don't matter for variance
- Simplifies computation enormously

Source: Cochran (1977), Chapter 11; Wolter (2007), Chapter 2

Three Methods to Estimate Variance

Taylor Linearization
Analytical approximation

Fast
Formula-based
Default in software

Jackknife
Delete-one-PSU

Robust
Detects outliers
Moderate speed

Bootstrap
Resample PSUs

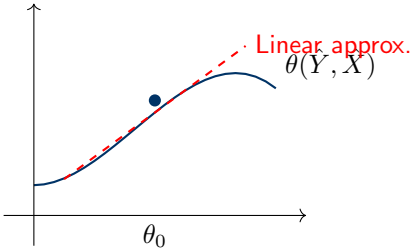
Flexible
Any statistic
Slow

All three methods:

- Account for stratification and clustering
- Use the ultimate cluster principle
- Give similar results for smooth statistics (means, totals)

Method 1: Taylor Series Linearization

Idea: Approximate complex statistics with linear functions.



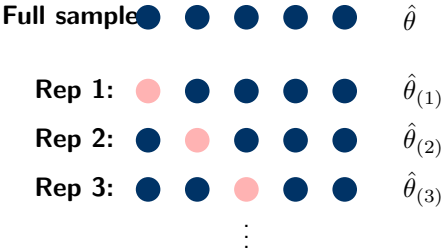
For a ratio $\hat{R} = \hat{Y}/\hat{X}$:

$$Var(\hat{R}) \approx \frac{1}{\bar{X}^2}\left[Var(\hat{Y}) + R^2 Var(\hat{X}) - 2R \cdot Cov(\hat{Y}, \hat{X})\right]$$

Uses first-order Taylor expansion around population values

Method 2: Jackknife Replication

Idea: Systematically delete one PSU at a time, observe variability.



Variance formula:

$$\widehat{Var}_{JK}(\hat{\theta}) = \frac{n-1}{n} \sum_{i=1}^{n} (\hat{\theta}_{(i)} - \hat{\theta})^2$$

Method 3: Bootstrap Replication

Idea: Resample PSUs with replacement, many times.

Original:	A	B	C	D	E	
Boot 1:	A	A	C	D	D	$\hat{\theta}_1^*$
Boot 2:	B	C	C	E	E	$\hat{\theta}_2^*$
Boot 3:	A	B	B	C	E	$\hat{\theta}_3^*$
	$\vdots$ (500 times)					

Variance formula:

$$\widehat{Var}_{Boot}(\hat{\theta}) = \frac{1}{B-1}\sum_{b=1}^{B}(\hat{\theta}_b^* - \bar{\theta}^*)^2$$

Lab 3.3 Results: Zambara GHS 2024

Zambara Case

Lindiwe runs all three variance estimation methods on mean household income...

Estimation target: Mean monthly household income

Sample: $n = 20,940$ households in 3,218 PSUs across 248 strata

Software	Commands
R	`svymean()`, `as.svrepdesign()`
STATA	`svy, vce(linearized/jackknife/bootstrap)`

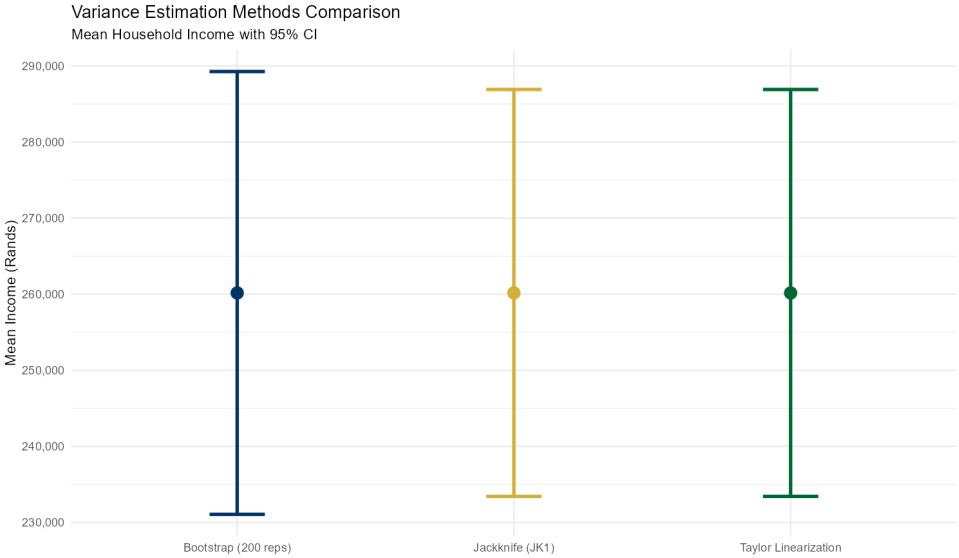
Variance Estimation: Method Comparison

Method	Mean (R)	SE (R)	CV	SE Ratio
Taylor Linearization	260,166	13,642	5.24%	1.0000
Jackknife (JK1)	260,166	13,643	5.24%	1.0001
Bootstrap (200 reps)	260,166	14,846	5.71%	1.0883
Naive ($s/\sqrt{n}$)	*260,166*	*3,305*	*1.27%*	*0.24*

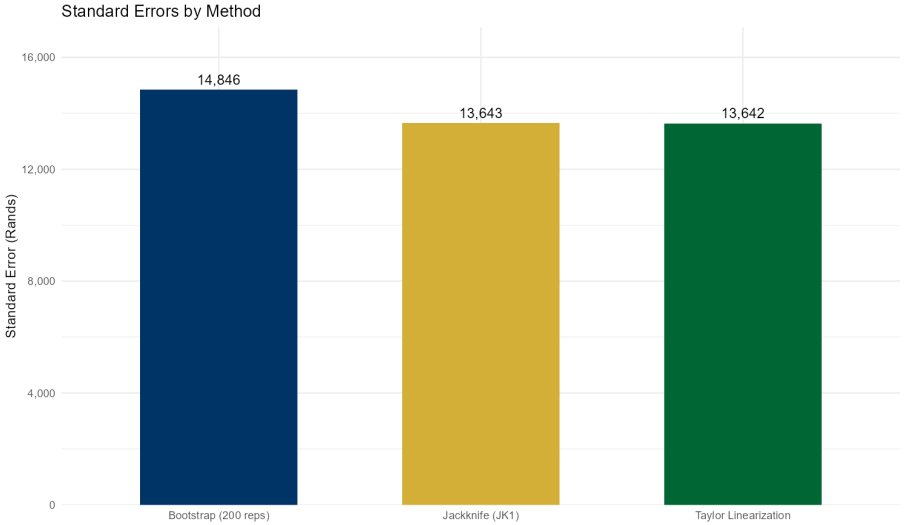
The naive SE underestimates true uncertainty by 76%!

DEFF $\approx (13,642/3,305)^2 = 17.0$

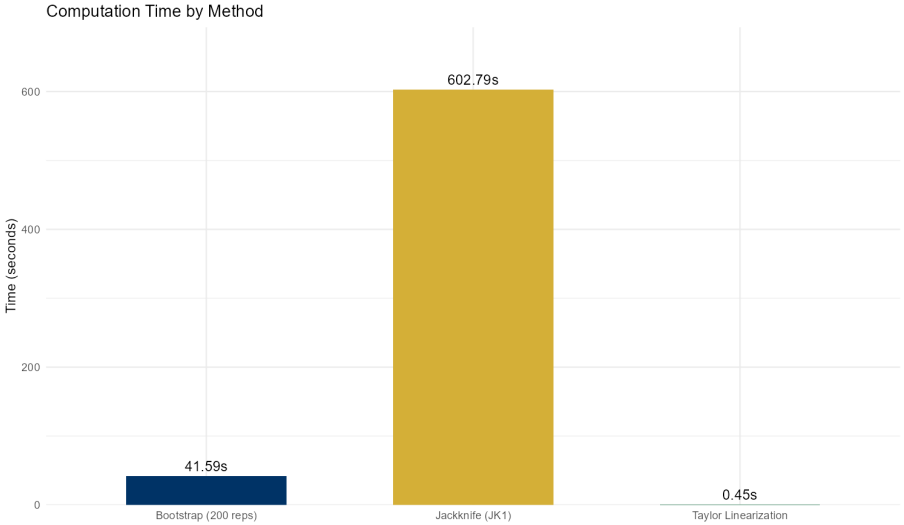
95% Confidence Intervals Comparison



Standard Errors by Method



Computation Time by Method



Why Do the Methods Agree?

For smooth statistics (means, totals, proportions):

- Taylor linearization is accurate (linear approximation works)
- Jackknife and bootstrap converge to same result
- Large sample sizes improve all approximations

When methods may diverge:

- Non-smooth statistics (medians, percentiles)
- Small samples or few PSUs
- Complex functions of multiple variables
- Statistics near boundaries (proportions near 0 or 1)

Best practice: Use Taylor for routine work; verify with replication methods for complex statistics or when results seem unusual.

Variance Varies by Domain

Province	Mean (R)	SE (R)	CV
KwaZulu-Natal	415,606	43,206	10.4%
Western Cape	354,566	47,423	13.4%
Free State	347,144	72,122	20.8%
Gauteng	243,107	23,988	9.9%
Eastern Cape	190,264	29,105	15.3%
Northern Cape	188,893	52,221	27.6%
North West	153,289	46,214	30.1%
Mpumalanga	152,888	31,815	20.8%
Limpopo	122,854	30,052	24.5%

Note: Gauteng has lowest CV (9.9%) despite moderate mean — larger sample, more homogeneous. North West has highest CV (30.1%) — smaller sample, high income variability.

Practical Implications

If you ignore design:

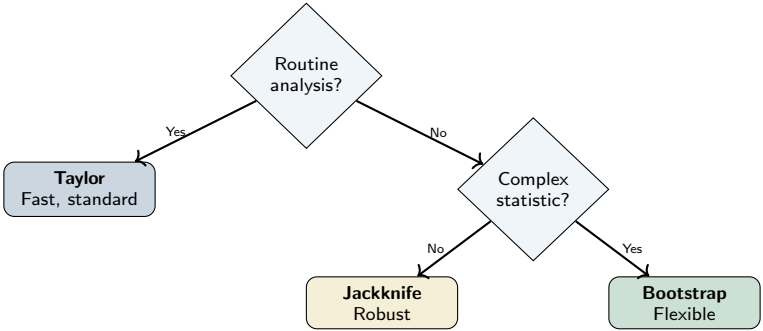
- SEs underestimated 4×
- CIs too narrow
- Spurious "significance"
- False precision
- Wrong policy conclusions

Correct approach:

- Account for clustering
- Account for stratification
- Use survey weights
- Report design-based SEs
- Interpret with appropriate uncertainty

Rule: Never use naive variance formulas with complex survey data!

Which Method Should You Use?



Situation	Recommended	Alternative
Means, totals, proportions	Taylor	Any
Medians, quantiles	Bootstrap	Jackknife
Ratios, regression coefficients	Taylor	Bootstrap
Quality assurance	Jackknife	Bootstrap

Special Issue: Singleton PSUs

Problem: Some strata have only 1 PSU ("lonely PSU")

- Cannot estimate within-stratum variance with $n_h = 1$
- Formula requires at least 2 PSUs per stratum

Solutions:

Option	Description
`certainty`	Treat as certainty PSU (contributes 0 to variance)
`adjust`	Center around grand mean instead of stratum mean
`average`	Use average variance from other strata
`combine`	Collapse singleton strata with similar strata

R: `options(survey.lonely.psu = "adjust")`
STATA: `svyset ..., singleunit(certainty)`

Degrees of Freedom in Complex Surveys

For confidence intervals and hypothesis tests:

$$df = (\text{number of PSUs}) - (\text{number of strata})$$

Zambara GHS:

$$df = 3,218 - 248 = 2,970$$

Why this matters:

- With large df, t-distribution $\approx$ Normal
- With small df (few PSUs), CIs should be wider
- Some software defaults to Normal — may undercover

Rule of thumb: With $df > 30$, Normal approximation is adequate. With $df < 30$, use t-distribution.

Software Commands Summary

R (survey package)

```r
# Taylor (default)
svymean(~income, design)

# Jackknife
des_jk <- as.svrepdesign(
  design, type = "JK1")
svymean(~income, des_jk)

# Bootstrap
des_boot <- as.svrepdesign(
  design, type = "bootstrap",
  replicates = 500)
svymean(~income, des_boot)
```

STATA

```stata
* Taylor (default)
svy: mean income

* Jackknife
svy, vce(jackknife): mean income

* Bootstrap
svy, vce(bootstrap, reps(500)): ///
    mean income
```

Key: Always define survey design first (`svydesign`/`svyset`) before estimation!

Common Mistakes to Avoid

1. **Using naive SEs** — Ignoring clustering/stratification
2. **Forgetting weights in variance** — Weighted mean but unweighted SE
3. **Wrong PSU variable** — Using household ID instead of cluster ID
4. **Ignoring singleton PSUs** — Software crashes or wrong results
5. **Too few bootstrap replicates** — Use 200+ for stable SEs
6. **Not checking convergence** — Bootstrap may need more iterations
7. **Mixing methods** — Point estimate from one, SE from another

Warning: Excel, basic SPSS, and most standard statistical functions do NOT account for complex designs. Use specialized survey software!

Module 3.3: Key Takeaways

1. **Naive variance fails:** s^2/n assumes independence — clusters violate this
2. **Clustering inflates variance:** Correlated observations = redundant information
3. **Stratification helps:** Removes between-stratum variance
4. **Design effect measures impact:** $DEFF = Var_{complex}/Var_{SRS}$
5. **Three methods available:** Taylor (fast), Jackknife (robust), Bootstrap (flexible)
6. **Methods agree for smooth statistics:** Means, totals, proportions
7. **Always use survey software:** `survey` package in R, `svy:` in STATA

Lindiwe now understands why her standard errors were 4× larger than expected!

Day 3 – Module 3.3 (Continued)

Taylor vs. Replication

A Deep Dive into Variance Estimation

Linearization • Jackknife • Bootstrap

Two Philosophies of Variance Estimation

Analytical (Taylor)

Philosophy: Derive a formula

- Use calculus to approximate
- Single pass through data
- Requires mathematical derivation
- Fast computation

Computational (Replication)

Philosophy: Simulate the process

- Perturb data, observe variability
- Multiple passes through data
- No formula needed
- Computationally intensive

Both approaches account for complex survey design and give similar results for smooth statistics.

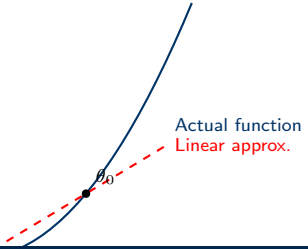
Taylor Linearization: The Big Idea

Core Concept

Approximate a **nonlinear** function with a **linear** one,
then use simple variance formulas for linear statistics.

Why this works:

- Near the true value, curves look like straight lines
- Linear functions have simple, known variance formulas
- Error from approximation is small for large samples



Taylor Series: A Quick Reminder

Taylor series expansion around point a:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \cdots$$

First-order (linear) approximation:

$$\boxed{f(x) \approx f(a) + f'(a)(x-a)}$$

For survey statistics:

- x = estimated quantities (random)
- a = population values (fixed)
- $f'(a)$ = slope at the true value

Named after Brook Taylor (1685–1731), English mathematician

Linearization for a Weighted Mean

The weighted mean is already linear:

$$\bar{y}_w = \frac{\sum_i w_i y_i}{\sum_i w_i} = \frac{\hat{Y}}{\hat{N}}$$

But it's a ratio! Apply the delta method:

$$Var\left(\frac{\hat{Y}}{\hat{N}}\right) \approx \frac{1}{\hat{N}^2}\left[Var(\hat{Y}) + \bar{y}_w^2 \cdot Var(\hat{N}) - 2\bar{y}_w \cdot Cov(\hat{Y}, \hat{N})\right]$$

Linearized variable:

$$z_i = \frac{y_i - \bar{y}_w}{\hat{N}/n}$$

Then: $Var(\bar{y}_w) \approx Var(\bar{z})$ using standard formulas.

Linearization for a Ratio

Estimate: $\hat{R} = \hat{Y}/\hat{X}$ (e.g., per-capita income)

Taylor expansion around (Y, X)**:**

$$\hat{R} - R \approx \frac{\partial R}{\partial Y}(\hat{Y} - Y) + \frac{\partial R}{\partial X}(\hat{X} - X)$$

Partial derivatives:

$$\frac{\partial R}{\partial Y} = \frac{1}{X}, \quad \frac{\partial R}{\partial X} = -\frac{Y}{X^2} = -\frac{R}{X}$$

Variance formula:

$$\boxed{Var(\hat{R}) \approx \frac{1}{X^2}\left[Var(\hat{Y}) + R^2 Var(\hat{X}) - 2R \cdot Cov(\hat{Y}, \hat{X})\right]}$$

Linearization in Practice

Step 1: Create linearized (pseudo) values for each PSU

$$z_{hi} = \sum_{j \in PSU_i} w_{hij} \cdot g(y_{hij})$$

where $g(\cdot)$ is the influence function

Step 2: Calculate variance between PSU totals within strata

$$\widehat{Var}(\hat{\theta}) = \sum_{h=1}^{H} \frac{n_h}{n_h - 1} \sum_{i=1}^{n_h} (z_{hi} - \bar{z}_h)^2$$

Step 3: Take square root for SE

$$SE(\hat{\theta}) = \sqrt{\widehat{Var}(\hat{\theta})}$$

Taylor Linearization: Advantages

Strengths:

1. **Speed** — Single pass, $O(n)$
2. **Deterministic** — Same data $\rightarrow$ same result
3. **No tuning** — No replicates to choose
4. **Memory efficient** — No weight matrices
5. **Default in software** — Well-tested

Zambara Lab 3.3:

Metric	Value
SE	R 13,642
Time	0.45 sec
Replicates	0

Fastest method
by far!

Taylor Linearization: Limitations

When Taylor may struggle:

1. **Nonsmooth statistics**
   - Medians, percentiles, mode
   - No derivative at the estimate
2. **Complex functions**
   - Gini coefficient, poverty measures
   - Difficult to derive linearized form
3. **Small samples**
   - Linear approximation less accurate
   - Higher-order terms matter
4. **Statistics near boundaries**
   - Proportions near 0 or 1
   - Variance estimates may be negative!

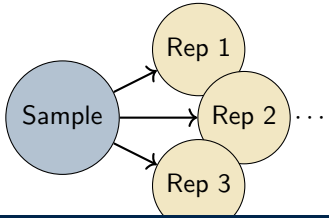
Replication Methods: The Big Idea

Core Concept

Mimic the sampling process to see how much estimates would vary if we drew many samples from the population.

The logic:

1. We have one sample, but want to know sampling variability
2. Create "pseudo-samples" by perturbing the original
3. Compute the statistic for each pseudo-sample
4. Variance of these estimates $\approx$ sampling variance



Jackknife: How It Works

Delete-one-group (JK1) for cluster samples:

1. For each PSU i in stratum h:
   - Delete PSU i
   - Reweight remaining PSUs: multiply by $\frac{n_h}{n_h - 1}$
   - Compute statistic $\hat{\theta}_{(hi)}$
2. Repeat for all PSUs (total $= \sum_h n_h$ replicates)
3. Calculate variance:

$$\boxed{\widehat{Var}_{JK}(\hat{\theta}) = \sum_{h=1}^{H} \frac{n_h - 1}{n_h} \sum_{i=1}^{n_h} (\hat{\theta}_{(hi)} - \hat{\theta})^2}$$

Note: The factor $\frac{n_h - 1}{n_h}$ corrects for the systematic nature of deletion.

Jackknife: Visual Representation

Stratum 1 (4 PSUs)

Full:	● ● ● ●	$\hat{\theta} = 260,166$
JK 1:	✕ ● ● ●	$\hat{\theta}_{(1)} = 258,420$
JK 2:	● ✕ ● ●	$\hat{\theta}_{(2)} = 261,890$
JK 3:	● ● ✕ ●	$\hat{\theta}_{(3)} = 259,100$
JK 4:	● ● ● ✕	$\hat{\theta}_{(4)} = 262,540$

$$Var = f(\hat{\theta}_{(1)}, \hat{\theta}_{(2)}, \hat{\theta}_{(3)}, \hat{\theta}_{(4)})$$

Jackknife: Replicate Weights

For replicate k (deleting PSU i in stratum h):

$$w_j^{(k)} = \begin{cases} 0 & \text{if } j \in \text{PSU } i \\ w_j \times \dfrac{n_h}{n_h - 1} & \text{if } j \in \text{stratum } h, j \notin \text{PSU } i \\ w_j & \text{otherwise} \end{cases}$$

Key properties:

- Deleted PSU gets weight 0
- Other PSUs in same stratum get inflated weights
- PSUs in other strata unchanged
- Sum of weights approximately preserved

Jackknife: Advantages

Strengths:

1. **Deterministic** — No random seed
2. **Detects outliers** — See which PSU drives variance
3. **Robust** — Works for many statistics
4. **No tuning** — Replicates = PSUs
5. **Exact for linear** — Matches Taylor exactly

Zambara Lab 3.3:

Metric	Value
SE	R 13,643
Time	602.79 sec
Replicates	3,218

SE virtually identical to Taylor!

Jackknife: Limitations

When Jackknife may struggle:

1. **Many PSUs = Slow**
   - Zambara: 3,218 PSUs = 3,218 replicates
   - Each replicate requires full estimation
2. **Nonsmooth statistics**
   - May overestimate variance for medians
   - Deleting one PSU can cause jumps
3. **Small strata**
   - With $n_h = 2$, deleting one PSU loses 50% of stratum
   - Variance estimate unstable
4. **Cannot assess distributional shape**
   - Only gives variance, not full distribution
   - No percentile CIs

Bootstrap: How It Works

Rescaled bootstrap for cluster samples:

1. For each replicate $b = 1, \ldots, B$:
   - Within each stratum h: resample $m_h = n_h - 1$ PSUs with replacement
   - Rescale weights to maintain variance properties
   - Compute statistic $\hat{\theta}_b^*$
2. After B replicates (typically 200–500):

$$\boxed{\widehat{Var}_{Boot}(\hat{\theta}) = \frac{1}{B-1} \sum_{b=1}^{B} (\hat{\theta}_b^* - \bar{\theta}^*)^2}$$

where $\bar{\theta}^* = \frac{1}{B} \sum_{b=1}^{B} \hat{\theta}_b^*$

Note: Resampling $n_h - 1$ instead of n_h is Rao-Wu adjustment for finite population.

Bootstrap: Visual Representation

Original PSUs:	A	B	C	D	E
Boot 1:	A	A	C	D	$\hat{\theta}_1^* = 257,890$
Boot 2:	B	B	D	E	$\hat{\theta}_2^* = 263,450$
Boot 3:	A	C	C	E	$\hat{\theta}_3^* = 259,120$
			$\vdots$		
Boot 200:	C	D	D	E	$\hat{\theta}_{200}^* = 261,780$

$$\downarrow$$

$$SE = SD(\hat{\theta}_1^*, \ldots, \hat{\theta}_{200}^*)$$

Bootstrap: Replicate Weights

For replicate b, PSU i in stratum h:

$$w_j^{(b)} = w_j \times \frac{n_h}{n_h - 1} \times \frac{m_{hi}^{(b)}}{1}$$

where $m_{hi}^{(b)}$ = number of times PSU i was selected in replicate b

Possible values of $m_{hi}^{(b)}$:

- 0 — PSU not selected (weight = 0)
- 1 — PSU selected once (weight $\approx$ original)
- 2 — PSU selected twice (weight $\approx 2\times$ original)
- 3+ — Rare but possible

The rescaling factor $\frac{n_h}{n_h - 1}$ accounts for resampling $n_h - 1$ PSUs.

Bootstrap: Advantages

Strengths:

1. **Universal** — Works for any statistic
2. **Distribution** — Get full sampling distribution
3. **Percentile CIs** — No normality assumption
4. **Flexible** — Easy to program
5. **Nonsmooth OK** — Medians, Gini, etc.

Zambara Lab 3.3:

Metric	Value
SE	R 14,846
Time	41.59 sec
Replicates	200

SE 8.8% higher
(random variation)

Bootstrap: Limitations

When Bootstrap may struggle:

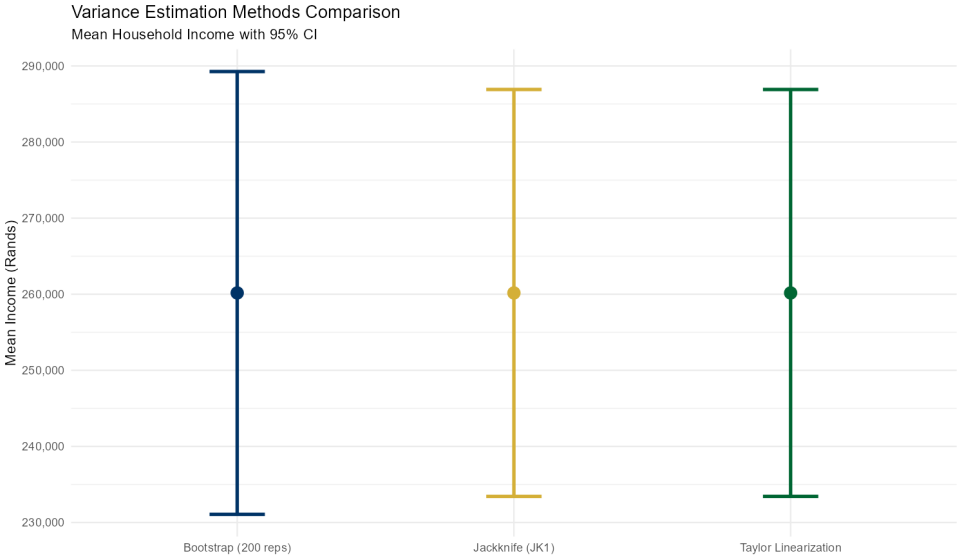
1. **Computationally intensive**
   - 500 replicates = 500× the computation
   - Can be slow for large datasets
2. **Random variation**
   - Results change with random seed
   - Need enough replicates for stability
3. **Choice of B**
   - Too few: unstable estimates
   - Too many: wasted computation
   - Rule: 200 for SE, 1000+ for CIs
4. **Small strata issues**
   - With $n_h = 2$: only resample 1 PSU
   - Limited variability captured

Head-to-Head: The Three Methods

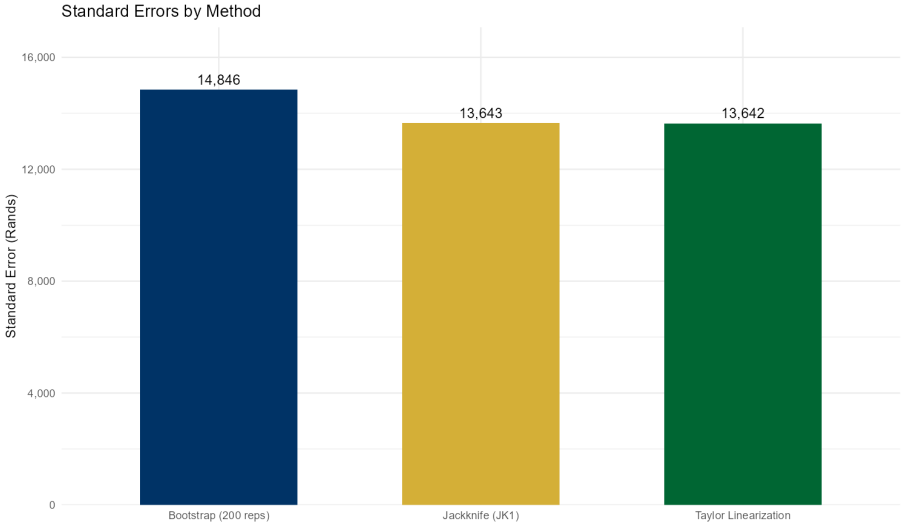
Feature	Taylor	Jackknife	Bootstrap
SE (Lab 3.3)	R 13,642	R 13,643	R 14,846
CV	5.24%	5.24%	5.71%
Time	0.45s	602.79s	41.59s
Replicates	0	3,218	200
Deterministic?	Yes	Yes	No
Formula needed?	Yes	No	No
Medians OK?	No	Caution	Yes
Percentile CI?	No	No	Yes

For the mean, all methods give similar SEs.
Taylor is fastest; Bootstrap is most flexible.

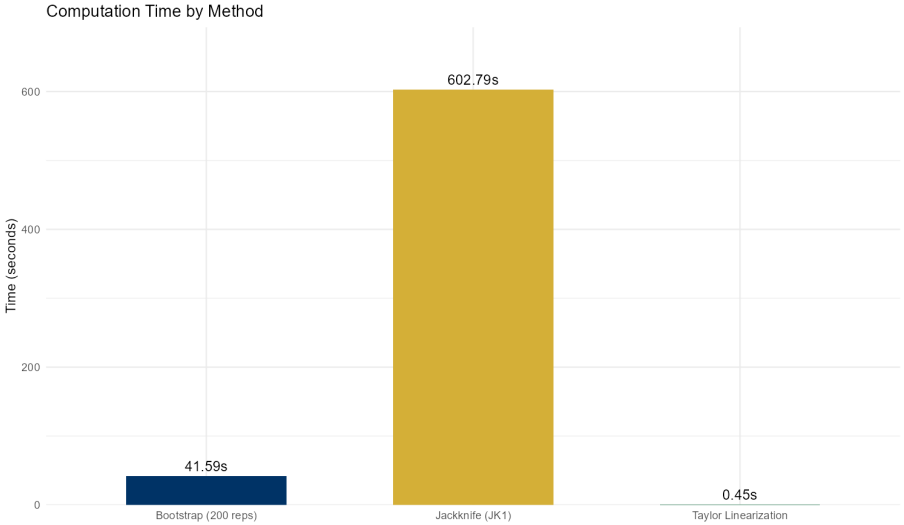
Lab 3.3 Results: Confidence Intervals



Lab 3.3 Results: Standard Errors



Lab 3.3 Results: Computation Time



Why Did the Methods Agree?

For the weighted mean (a smooth statistic):

1. **Taylor linearization is exact for linear statistics**
   - Mean is a ratio, but linear approximation is excellent
2. **Jackknife converges to Taylor**
   - For differentiable statistics, JK $\rightarrow$ Taylor as $n \rightarrow \infty$
3. **Bootstrap converges to truth**
   - With enough replicates, mimics true sampling distribution
   - 200 replicates: some Monte Carlo error remains
4. **Large sample size**
   - $n = 20,940$ households
   - All asymptotic approximations work well

When Methods Would Diverge

Scenario 1: Median income

- Taylor: Requires density estimation (complex)
- Jackknife: May overestimate variance (jumps)
- Bootstrap: Most reliable

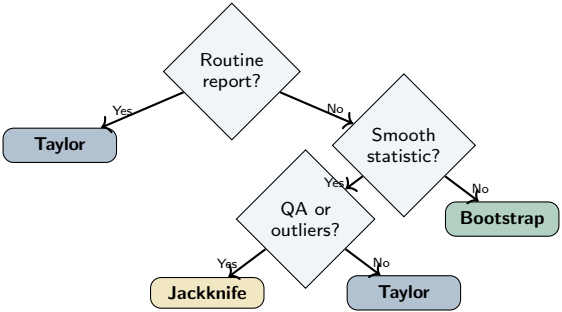
Scenario 2: Small sample (e.g., $n = 200$)

- Taylor: Approximation error larger
- Jackknife: Few replicates, unstable
- Bootstrap: Also affected, but can use more replicates

Scenario 3: Proportion near 0% or 100%

- Taylor: May give negative variance!
- Jackknife: Similar issues
- Bootstrap: Naturally bounded

Choosing a Method: Decision Flowchart



Rule of thumb: Start with Taylor; use replication for validation or complex statistics.

Practical Recommendations

1. **For routine analysis:** Use Taylor (default)
   - Fast, deterministic, well-tested
   - R: `svymean()`, STATA: `svy: mean`
2. **For quality assurance:** Run Jackknife occasionally
   - Verify Taylor results, detect influential PSUs
   - Flag if SE differs by $> 10\%$
3. **For complex statistics:** Use Bootstrap
   - Medians, percentiles, Gini, poverty measures
   - Use 500+ replicates; set seed for reproducibility
4. **For public-use files:** Provide replicate weights
   - Allows users to compute correct SEs without design details
   - Common: 80–200 replicates stored in file

Software Syntax: Side-by-Side

R (survey package)

```r
# Setup
library(survey)
des <- svydesign(id=~psu,
  strata=~stratum,
  weights=~wgt, data=df)

# Taylor (default)
svymean(~income, des)

# Jackknife
des_jk <- as.svrepdesign(
  des, type="JK1")
svymean(~income, des_jk)

# Bootstrap
des_boot <- as.svrepdesign(
  des, type="bootstrap",
  replicates=500)
svymean(~income, des_boot)
```

STATA

```stata
* Setup
svyset psu [pw=wgt], ///
  strata(stratum)

* Taylor (default)
svy: mean income

* Jackknife
svy, vce(jackknife): ///
  mean income

* Bootstrap
svy, vce(bootstrap, ///
  reps(500)): mean income
```

Module 3.3: Key Takeaways

1. **Taylor linearization:** Fast, analytical, uses calculus
   - Best for routine means, totals, proportions
2. **Jackknife:** Delete one PSU at a time, observe variability
   - Deterministic, good for QA, detects outliers
3. **Bootstrap:** Resample PSUs, compute distribution
   - Most flexible, essential for complex statistics
4. **For smooth statistics:** All methods agree
   - Zambara: Taylor SE = R 13,642, JK SE = R 13,643
5. **Default to Taylor:** Fast and accurate for most cases
6. **Use replication for:** Validation, medians, percentiles, Gini

Next: Module 3.4 — Calibration and Post-Stratification

Day 3 – Module 3.4

Mastering Replication

A Technical Deep Dive

Jackknife • BRR • Bootstrap • Replicate Weights

The Replication Philosophy

Core Principle

Estimate variance by **repeatedly perturbing** the sample and observing how the statistic changes.

General formula for all replication methods:

$$\boxed{\widehat{Var}_{rep}(\hat{\theta}) = c \sum_{r=1}^{R} (\hat{\theta}^{(r)} - \hat{\theta}_0)^2}$$

Where:

- R = number of replicates
- $\hat{\theta}^{(r)}$ = estimate from replicate r
- $\hat{\theta}_0$ = reference estimate (full sample or mean of replicates)
- c = method-specific constant

Three Families of Replication Methods

Jackknife
Delete-one
Systematic

$R = n_{PSU}$
Deterministic
Linear statistics

BRR
Balanced halves
Orthogonal

$R =$ small (e.g., 44)
Deterministic
Paired PSUs

Bootstrap
Resample
Random

$R = 200$–1000
Random
Any statistic

All methods:

- Account for stratification and clustering
- Can be stored as replicate weight columns
- Enable variance estimation without design details

Jackknife Family: Overview

Origin: Quenouille (1949), Tukey (1958) — "jackknife" like a versatile tool

Core idea: Systematically delete one unit (or group) at a time

Variant	Deletion Unit	Use Case
JK1	One PSU	Unstratified or stratified
JKn	One PSU per stratum	Stratified designs
JK2	Paired PSUs	2 PSUs per stratum

Key property: Number of replicates = Number of PSUs

$$R_{JK} = \sum_{h=1}^{H} n_h = \text{Total PSUs in sample}$$

JK1: Delete-One-PSU Method

Algorithm:

1. For replicate r corresponding to PSU i in stratum h:
   - Set weights to 0 for all units in PSU i
   - Inflate weights for remaining PSUs in stratum h
2. Compute $\hat{\theta}^{(r)}$ using modified weights
3. Repeat for all PSUs ($r = 1, \ldots, R$)

Variance formula:

$$\widehat{Var}_{JK1}(\hat{\theta}) = \sum_{h=1}^{H} \frac{n_h - 1}{n_h} \sum_{i=1}^{n_h} (\hat{\theta}^{(hi)} - \hat{\theta})^2$$

Constant c: Varies by stratum — $c_h = \frac{n_h - 1}{n_h}$

JK1: Constructing Replicate Weights

For unit j in PSU i, stratum h, replicate r (deleting PSU i' in stratum h'):

$$w_j^{(r)} = \begin{cases} 0 & \text{if } i = i' \text{ (deleted PSU)} \\ w_j \times \dfrac{n_h}{n_h - 1} & \text{if } h = h', i \neq i' \text{ (same stratum)} \\ w_j & \text{if } h \neq h' \text{ (different stratum)} \end{cases}$$

Example: Stratum with 5 PSUs

	PSU 1	PSU 2	PSU 3	PSU 4	PSU 5
Full sample	w	w	w	w	w
Replicate 1	0	$1.25w$	$1.25w$	$1.25w$	$1.25w$
Replicate 2	$1.25w$	0	$1.25w$	$1.25w$	$1.25w$
Replicate 3	$1.25w$	$1.25w$	0	$1.25w$	$1.25w$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

JKn: Stratified Jackknife

Used when: Design has many strata with varying numbers of PSUs

Difference from JK1:

- JK1: Single formula for all strata
- JKn: Stratum-specific adjustment factors

Variance formula:

$$\widehat{Var}_{JKn}(\hat{\theta}) = \sum_{h=1}^{H} \frac{n_h - 1}{n_h} \sum_{i=1}^{n_h} (\hat{\theta}^{(hi)} - \bar{\theta}_h)^2$$

where $\bar{\theta}_h = \frac{1}{n_h} \sum_{i=1}^{n_h} \hat{\theta}^{(hi)}$ (mean of replicates in stratum h)

Key distinction: Centers around stratum mean $\bar{\theta}_h$ instead of full-sample $\hat{\theta}$

Balanced Repeated Replication (BRR)

Origin: McCarthy (1966, 1969) for U.S. Current Population Survey

Requirement: Exactly 2 PSUs per stratum (paired design)

Core idea:

- Split each stratum into two half-samples
- Use **Hadamard matrix** to ensure balance
- Much fewer replicates than Jackknife!

Stratum 1:	A	B		Half 1: A, A, B
Stratum 2:	A	B	→	Half 2: B, B, A
Stratum 3:	A	B		Half 3: A, B, A
				Half 4: B, A, B

BRR: The Hadamard Matrix

Hadamard matrix $\mathbf{H}$: Orthogonal matrix with entries ± 1

Example for 4 strata:

$$\mathbf{H}_4 = \begin{pmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{pmatrix}$$

Interpretation:

- Each row = one replicate
- Each column = one stratum
- $+1$ = select PSU A (first unit)
- -1 = select PSU B (second unit)

Balance property: Each PSU appears in exactly half the replicates

BRR: Constructing Replicate Weights

For unit j in PSU i (first or second in stratum h), replicate r:

$$w_j^{(r)} = w_j \times \delta_{hi}^{(r)} \times 2$$

where $\delta_{hi}^{(r)} = \begin{cases} 1 & \text{if PSU selected in replicate } r \\ 0 & \text{otherwise} \end{cases}$

Variance formula:

$$\widehat{Var}_{BRR}(\hat{\theta}) = \frac{1}{R} \sum_{r=1}^{R} (\hat{\theta}^{(r)} - \hat{\theta})^2$$

Number of replicates:

- Minimum: $R \geq H$ (number of strata)
- Hadamard matrices exist for $R = 4, 8, 12, 16, 20, 24, \ldots$
- Typically much smaller than Jackknife ($R_{BRR} \ll R_{JK}$)

BRR: Worked Example

Design: 3 strata, 2 PSUs each (6 PSUs total)

Replicate	Stratum 1 PSU A	Stratum 1 PSU B	Stratum 2 PSU A	Stratum 2 PSU B	Stratum 3 PSU A	Stratum 3 PSU B
Full	w	w	w	w	w	w
Rep 1 $(+,+,+)$	$2w$	0	$2w$	0	$2w$	0
Rep 2 $(+,-,+)$	$2w$	0	0	$2w$	$2w$	0
Rep 3 $(+,+,-)$	$2w$	0	$2w$	0	0	$2w$
Rep 4 $(+,-,-)$	$2w$	0	0	$2w$	0	$2w$

Key observations:

- Only 4 replicates (vs. 6 for Jackknife)
- Each PSU selected exactly 2 times (balanced)
- Weights doubled when selected (compensates for half-sample)

Fay's BRR: A Gentler Version

Problem with standard BRR: Weights go to 0 for non-selected PSUs

Fay's modification: Use perturbation factor $\rho \in (0, 1)$

$$w_j^{(r)} = w_j \times \begin{cases} 2 - \rho & \text{if PSU selected} \\ \rho & \text{if PSU not selected} \end{cases}$$

Common choice: $\rho = 0.5$

- Selected PSU: $w \times 1.5$
- Non-selected PSU: $w \times 0.5$

Adjusted variance:

$$\widehat{Var}_{Fay}(\hat{\theta}) = \frac{1}{R(1-\rho)^2} \sum_{r=1}^{R} (\hat{\theta}^{(r)} - \hat{\theta})^2$$

Bootstrap: Overview

Origin: Efron (1979) — general-purpose resampling

Adaptation for surveys: Rao & Wu (1988), McCarthy & Snowden (1985)

Core idea:

- Resample PSUs **with replacement** within strata
- Random process (not systematic like JK or BRR)
- Flexible — works for **any** statistic

Key parameters:

- B = number of bootstrap replicates (typically 200–1000)
- Larger B = more stable variance estimate
- Trade-off: precision vs. computation time

Variance formula:

$$\widehat{Var}_{Boot}(\hat{\theta}) = \frac{1}{B-1}\sum_{b=1}^{B}(\hat{\theta}^{(b)} - \bar{\theta}^*)^2$$

Rescaled Bootstrap for Surveys

Problem: Naive bootstrap underestimates variance for finite populations

Rao-Wu Rescaled Bootstrap:

1. Within each stratum h, resample $m_h = n_h - 1$ PSUs with replacement
2. Rescale weights to correct variance

Why $n_h - 1$ instead of n_h?

- Resampling n_h PSUs underestimates variance
- Factor $\frac{n_h}{n_h - 1}$ corrects the bias
- Same logic as $(n - 1)$ in sample variance formula

Replicate weight for unit j in PSU i, stratum h, replicate b:

$$w_j^{(b)} = w_j \times \frac{n_h}{n_h - 1} \times m_i^{(b)}$$

where $m_i^{(b)}$ = number of times PSU i selected in replicate b

Bootstrap: Weight Multipliers

For a PSU in a stratum with n_h PSUs, weight multiplier distribution:

Times Selected (m)	Probability	Multiplier
0	$\approx 37\%$	0
1	$\approx 37\%$	$\frac{n_h}{n_h-1}$
2	$\approx 18\%$	$\frac{2n_h}{n_h-1}$
3	$\approx 6\%$	$\frac{3n_h}{n_h-1}$
4+	$\approx 2\%$	$\frac{m \cdot n_h}{n_h-1}$

Note: As $n_h \to \infty$, probability of 0 selections $\to e^{-1} \approx 0.368$

Unlike BRR/JK where each PSU has fixed role, bootstrap weights are random!

Replication Methods: Detailed Comparison

Feature	JK1/JKn	BRR	Fay BRR	Bootstrap
Design requirement	Any	2 PSU/stratum	2 PSU/stratum	Any
Number of replicates	$\sum n_h$	$\geq H$	$\geq H$	200–1000
Deterministic	Yes	Yes	Yes	No
Zero weights	Yes	Yes	No	Yes
Medians/percentiles	Caution	Caution	Better	Best
Percentile CIs	No	No	No	Yes
Storage (200 strata)	3000+ cols	200 cols	200 cols	500 cols

Computational cost:

- JK: $O(n_{PSU})$ replicates — can be slow with many PSUs
- BRR: $O(H)$ replicates — very efficient
- Bootstrap: $O(B)$ replicates — user-controlled

The Lonely PSU Problem

Singleton stratum: Only 1 PSU in a stratum

Why it's a problem:

$$Var_h = \frac{n_h}{n_h - 1} \sum_{i=1}^{n_h} (\cdot)^2 \quad \Rightarrow \quad \frac{1}{1-1} = \frac{1}{0} = \text{undefined!}$$

Causes:

- Design flaw (should always sample ≥ 2 PSUs)
- Non-response eliminated one PSU
- Data errors during processing

Lab 3.4 Results: Simulated 1 singleton stratum in GHS 2024

Method	SE (R)	SE Ratio
Baseline (no singleton)	13,642	1.0000
Certainty	13,643	1.0001
Adjust (Centered)	13,647	1.0004
Average	13,670	1.0021

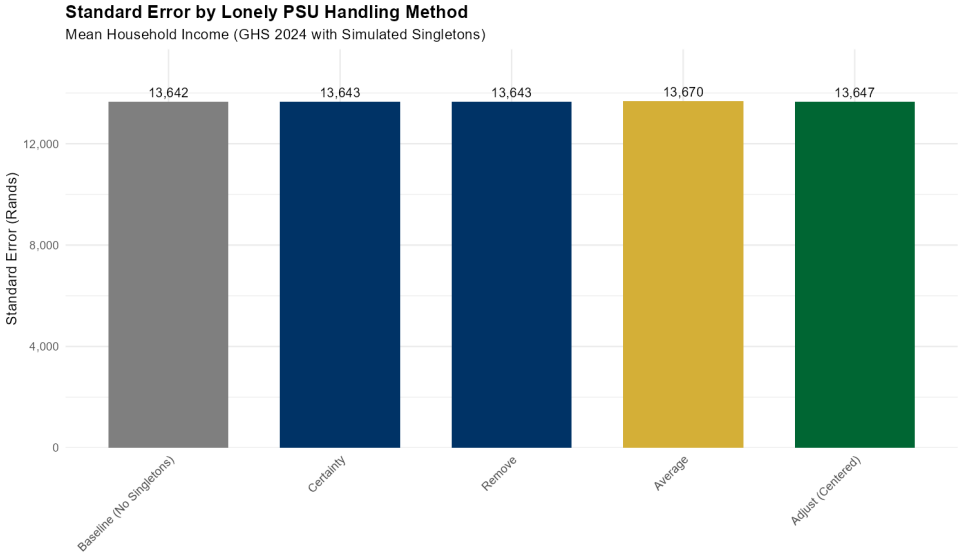
Handling Lonely PSUs: Software Options

R Option	STATA Option	Treatment
`"fail"`	`singleunit(missing)`	Error / missing SE
`"certainty"`	`singleunit(certainty)`	Singleton contributes 0
`"adjust"`	`singleunit(centered)`	Center around grand mean
`"average"`	—	Use average from other strata
`"remove"`	—	Remove from variance calc
—	`singleunit(scaled)`	Scaled centered method

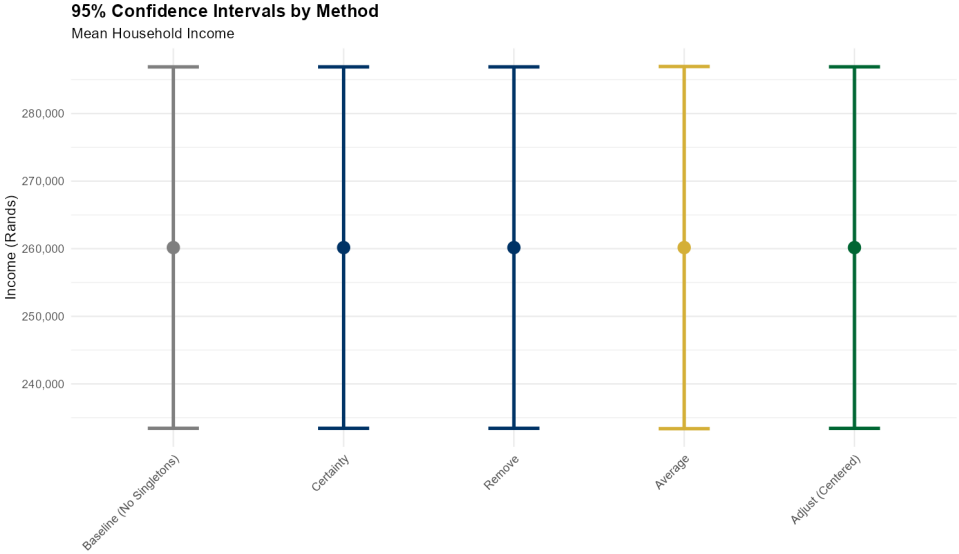
Recommendations:

1. **Prevention:** Design with ≥ 2 PSUs per stratum
2. **Conservative:** Use `adjust/centered` (inflates SE)
3. **Document:** Report singleton handling in methods

Lab 3.4: Standard Errors by Lonely PSU Method



Lab 3.4: Confidence Intervals by Method



Replicate Weights in Public-Use Files

Why provide replicate weights?

- Users can compute correct SEs without knowing design details
- Protects confidentiality (PSU/stratum IDs not released)
- Standardizes variance estimation across users

Typical file structure:

ID	PWGT	REPWT1	REPWT2	REPWT3	$\cdots$	REPWT80
1	1523	1827	0	1523	$\cdots$	3046
2	892	892	1784	892	$\cdots$	0
3	2105	2105	2105	0	$\cdots$	2105
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$

Common standards:

- U.S. Census: 80 BRR replicates (Fay's method, $\rho = 0.5$)
- Eurostat: 1000 bootstrap replicates

Using Replicate Weights in R

```r
# If replicate weights already exist in data
library(survey)

# Create design from replicate weights
des_rep <- svrepdesign(
  data = mydata,
  weights = ~PWGT,              # Main weight
  repweights = "REPWT[0-9]+", # Pattern for replicate cols
  type = "Fay",                 # Method used to create
  rho = 0.5,                    # Fay's rho
  combined.weights = TRUE       # Replicates include main weight
)

# Estimate with correct SEs
svymean(~income, des_rep)
svytotal(~employed, des_rep)
svyquantile(~income, des_rep, quantiles = 0.5)
```

Key: Must specify correct `type` and `rho` to match how weights were created!

Using Replicate Weights in STATA

```
* If replicate weights exist in data
* Method 1: Specify at svyset
svyset [pw=PWGT], brrweight(REPWT*) ///
    vce(brr) fay(0.5)

svy: mean income
svy: total employed
svy: tabulate region

* Method 2: Use svy brr directly
svy brr, rw(REPWT*) fay(0.5): mean income

* For bootstrap weights
svyset [pw=PWGT], bsrweight(REPWT*) vce(bootstrap)
svy: mean income
```

Note: `brrweight` for BRR, `bsrweight` for bootstrap

Creating Replicate Weights in R

```
library(survey)

# Start with standard design
des <- svydesign(
  id = ~psu, strata = ~stratum,
  weights = ~house_wgt, data = ghs
)

# Create Jackknife replicates
des_jk <- as.svrepdesign(des, type = "JK1")

# Create BRR replicates (requires 2 PSUs/stratum)
des_brr <- as.svrepdesign(des, type = "BRR")

# Create Fay BRR
des_fay <- as.svrepdesign(des, type = "Fay", fay.rho = 0.5)

# Create Bootstrap replicates
des_boot <- as.svrepdesign(des, type = "bootstrap",
                           replicates = 500)

# Extract replicate weights matrix
rep_weights <- weights(des_boot, type = "analysis")
```

Creating Replicate Weights in STATA

```
* Standard survey setup
svyset psu [pw=house_wgt], strata(stratum)

* Create BRR weights (requires 2 PSUs/stratum)
bsweights brrwt, reps(100) n(-1) seed(12345)

* Create bootstrap weights
bsweights bswt, reps(500) n(-1) seed(12345)

* Alternative: use brrweight command
* (requires balanced design)

* Save replicate weights for public use file
keep id house_wgt brrwt* bswt*
save "public_use_weights.dta", replace

* Verify weights work
svyset [pw=house_wgt], brrweight(brrwt*) vce(brr)
svy: mean income
```

Note: `bsweights` is user-written; install via `ssc install bsweights`

How Many Replicates?

For Bootstrap:

Purpose	**Minimum B**	**Recommended B**
Standard errors	50	200
95% CIs (percentile)	500	1000
99% CIs (percentile)	1500	2000
Publication quality	500	1000+

Monte Carlo error in SE estimate:

$$CV(\widehat{SE}) \approx \frac{1}{\sqrt{2B}}$$

- $B = 50$: CV $\approx 10\%$
- $B = 200$: CV $\approx 5\%$
- $B = 500$: CV $\approx 3\%$

Bootstrap Percentile Confidence Intervals

Advantage of bootstrap: Non-symmetric CIs without normality assumption

Percentile method:

1. Compute $\hat{\theta}^{(b)}$ for $b = 1, \ldots, B$
2. Sort bootstrap estimates
3. 95% CI: $[\hat{\theta}^*_{(0.025B)}, \hat{\theta}^*_{(0.975B)}]$

Example: With $B = 1000$

- Lower bound: 25th smallest bootstrap estimate
- Upper bound: 975th smallest bootstrap estimate

When to use:

- Skewed distributions (e.g., income)
- Statistics near boundaries (proportions near 0 or 1)
- Non-smooth statistics (medians, percentiles)

Quality Assurance: Verifying Replication

How to check if replicate weights are correct:

1. **Compare to Taylor:** For means/totals, results should match
   - Tolerance: $\pm 5\%$ for SE
   - Larger differences suggest errors
2. **Check weight sums:**
   - Sum of replicate weights $\approx$ sum of main weights
   - Large deviations indicate problems
3. **Examine replicate distribution:**
   - JK/BRR: Weights should be 0 or inflated
   - Bootstrap: Should follow expected distribution
4. **Test on known quantities:**
   - Population totals from admin data
   - Should be within CI range

Common Pitfalls in Replication

1. **Wrong method specification**
   - Using BRR formula with JK weights (or vice versa)
   - Forgetting Fay's ρ adjustment
2. **Insufficient replicates (bootstrap)**
   - 50 replicates: SE estimates unstable
   - Use 200+ for routine work, 500+ for CIs
3. **Not setting random seed**
   - Bootstrap results non-reproducible
   - Always set seed for replicability
4. **Ignoring lonely PSUs**
   - Causes errors or missing SEs
   - Set appropriate handling option
5. **Mixing weight types**
   - Using calibrated weights with uncalibrated replicates
   - Replicates must reflect same adjustments

Module 3.4: Key Takeaways

1. **Three replication families:**
   - Jackknife: Delete-one, $R = n_{PSU}$, deterministic
   - BRR: Balanced halves, $R \approx H$, requires paired PSUs
   - Bootstrap: Random resampling, $R = 200$–1000, flexible
2. **Replicate weights enable:**
   - Variance estimation without design details
   - Confidentiality protection in public-use files
3. **Handle lonely PSUs:**
   - Prevent in design (≥ 2 PSUs per stratum)
   - Use `adjust/centered` for conservative SEs
4. **Choose method based on:**
   - Design structure (2 PSUs/stratum $\rightarrow$ BRR)
   - Statistic complexity (medians $\rightarrow$ Bootstrap)
   - Computational constraints

Next: Module 3.5 — Calibration and Post-Stratification

Day 3 – Module 3.4 Continued

Troubleshooting Variance

A Problem-Solving Guide

Lonely PSUs • Degrees of Freedom • Diagnostic Toolkit

Zambara Case: The Mysterious Missing Standard Error

Zambara Case

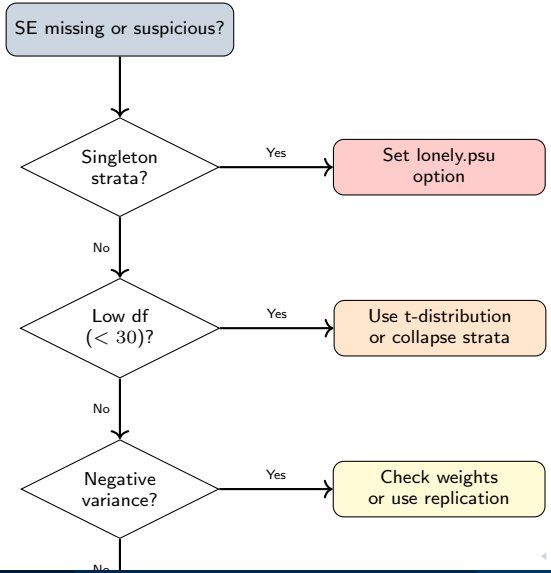
Lindiwe runs her first analysis of the Household Living Conditions Survey. The mean income estimate looks reasonable, but where the standard error should appear, she sees only: `NA` (in R) or `.` (in STATA).

"The estimate is there, but I can't report precision without a standard error!"

Common causes of missing/problematic SEs:

1. Singleton strata (lonely PSUs)
2. Insufficient degrees of freedom
3. Negative variance estimates
4. Computational overflow/underflow
5. Design specification errors

Variance Troubleshooting Flowchart



Problem #1: The Lonely PSU

Definition

A **singleton stratum** (lonely PSU) occurs when a stratum contains only one Primary Sampling Unit.

Why it breaks variance estimation:

$$\widehat{Var}_h = \frac{n_h}{n_h - 1} \sum_{i=1}^{n_h} (y_{hi} - \bar{y}_h)^2$$

When $n_h = 1$:

- Denominator: $n_h - 1 = 1 - 1 = 0 \quad \Rightarrow$ **Division by zero!**
- Numerator: $(y_{h1} - \bar{y}_h)^2 = (y_{h1} - y_{h1})^2 = 0$
- No information about within-stratum variability

Result: Software returns `NA`, `.`, or throws an error

How Singleton Strata Occur

Design-Phase Causes:

1. Only 1 PSU exists in stratum
   - Remote island province
   - Very small domain
2. Certainty selection
   - Large PSU sampled with prob=1
   - Self-representing unit
3. Over-stratification
   - Too many strata created
   - Province $\times$ urban $\times$ income

Post-Design Causes:

1. Non-response
   - One PSU refused entirely
   - Conflict/disaster areas
2. Data processing errors
   - Miscoded stratum IDs
   - Merge/append mistakes
3. Subsetting data
   - Domain with sparse strata
   - Subpopulation analysis

Detecting Singleton Strata in R

```
# Method 1: Check before creating design
library(dplyr)
psu_counts <- ghs %>%
  group_by(stratum) %>%
  summarise(n_psu = n_distinct(psu), .groups = "drop")

# Find singletons
singletons <- psu_counts %>% filter(n_psu == 1)
print(singletons)

# Method 2: After creating design, check degf()
library(survey)
des <- svydesign(id = ~psu, strata = ~stratum,
                 weights = ~house_wgt, data = ghs)

# Degrees of freedom by stratum
# Total df = sum(n_h - 1) across strata
cat("Total degrees of freedom:", degf(des), "\n")

# Method 3: Try estimation and catch error
tryCatch({
  svymean(~income, des)
}, error = function(e) {
  message("Singleton detected: ", e$message)
})
```

Detecting Singleton Strata in STATA

```
* Method 1: Tabulate PSUs per stratum
bysort stratum: gen psu_tag = (_n == 1) if psu != .
bysort stratum: egen n_psu = total(psu_tag)

* List singleton strata
list stratum n_psu if n_psu == 1 & psu_tag == 1, noobs

* Count singletons
count if n_psu == 1 & psu_tag == 1
display "Number of singleton strata: " r(N)

* Method 2: Use svydescribe after svyset
svyset psu [pw=house_wgt], strata(stratum)
svydescribe

* Look for "1 stratum with 1 sampling unit" message
* Or strata listed with only 1 unit

* Method 3: Check singleunit setting effect
svy, singleunit(missing): mean income
* If SE is missing (.), singletons are affecting estimate
```

Lab 3.4: Simulating Singleton Strata

Objective: Create and fix singleton stratum scenarios

Setup:

- Data: GHS 2024 Household File (20,940 households)
- Original design: 248 strata, 3,218 PSUs
- Simulation: Created 1 artificial singleton stratum

Singleton stratum created:

Stratum	PSUs	Households	Status
9999 (simulated)	1	3	Singleton

Analysis variable: Total monthly household income (`totmhinc`)

Lab 3.4: Demonstrating the Error

R code with default settings:

```
options(survey.lonely.psu = "fail")  # Default

des <- svydesign(id = ~psu, strata = ~stratum,
                 weights = ~house_wgt, data = ghs_sim)

svymean(~totmhinc, des, na.rm = TRUE)
```

Error message:

```
Error in onestrat(x[index, , drop = FALSE], clusters[index], nPSU[index][1], :  Stratum (9999) has only
one PSU at stage 1
```

STATA equivalent:

```
svyset psu [pw=house_wgt], strata(stratum) singleunit(missing)
svy: mean totmhinc
* Result: SE displayed as "." (missing)
```

Lonely PSU Solutions: Four Approaches

Certainty
Contributes 0 to variance

Underestimates

Adjust
Center around grand mean

Conservative

Average
Use variance from other strata

Moderate

Remove
Exclude from variance calc

Risky if different

Software equivalents:

R	STATA
`options(survey.lonely.psu = "certainty")`	`svyset ..., singleunit(certainty)`
`options(survey.lonely.psu = "adjust")`	`svyset ..., singleunit(centered)`
`options(survey.lonely.psu = "average")`	(not available)

Solution 1: Certainty

Assumption: PSU was selected with probability 1 (self-representing)

Treatment: Singleton stratum contributes **zero** to variance

R Implementation:

```
options(survey.lonely.psu =
        "certainty")

des <- svydesign(
  id = ~psu,
  strata = ~stratum,
  weights = ~house_wgt,
  data = ghs_sim
)

svymean(~totmhinc, des,
        na.rm = TRUE)
```

STATA Implementation:

```
svyset psu [pw=house_wgt], ///
  strata(stratum) ///
  singleunit(certainty)

svy: mean totmhinc
```

Caution

Use **only** when the PSU truly was a certainty selection. Otherwise, variance will be **underestimated**.

Solution 2: Adjust (Centered)

Assumption: Singleton's deviation from grand mean reflects variability

Treatment: Center around **grand mean** instead of stratum mean

R Implementation:

```
options(survey.lonely.psu =
        "adjust")

des <- svydesign(
  id = ~psu,
  strata = ~stratum,
  weights = ~house_wgt,
  data = ghs_sim
)

svymean(~totmhinc, des,
        na.rm = TRUE)
```

STATA Implementation:

```
svyset psu [pw=house_wgt], ///
  strata(stratum) ///
  singleunit(centered)

svy: mean totmhinc
```

Recommended

Most **conservative** approach. Typically **increases** SE, reducing risk of false precision. Good default choice when uncertain.

Solution 3: Average (R Only)

Assumption: Singleton stratum has similar variance to other strata

Treatment: Use **average variance** from non-singleton strata

```
options(survey.lonely.psu = "average")

des <- svydesign(
  id = ~psu,
  strata = ~stratum,
  weights = ~house_wgt,
  data = ghs_sim
)

mean_result <- svymean(~totmhinc, des, na.rm = TRUE)
```

When to use:

- Singleton occurred randomly (e.g., non-response)
- Singleton stratum is similar to others
- Not available in STATA — use `centered` as alternative

Solution 4: Remove (R Only)

Assumption: Singleton stratum is negligible for variance

Treatment: **Exclude** singleton from variance calculation entirely

```
options(survey.lonely.psu = "remove")

des <- svydesign(
  id = ~psu,
  strata = ~stratum,
  weights = ~house_wgt,
  data = ghs_sim
)

mean_result <- svymean(~totmhinc, des, na.rm = TRUE)
```

Warning

Risky if singleton stratum is systematically different from others. May **underestimate** variance if singleton has high variability.

Lab 3.4 Results: Comparing All Methods

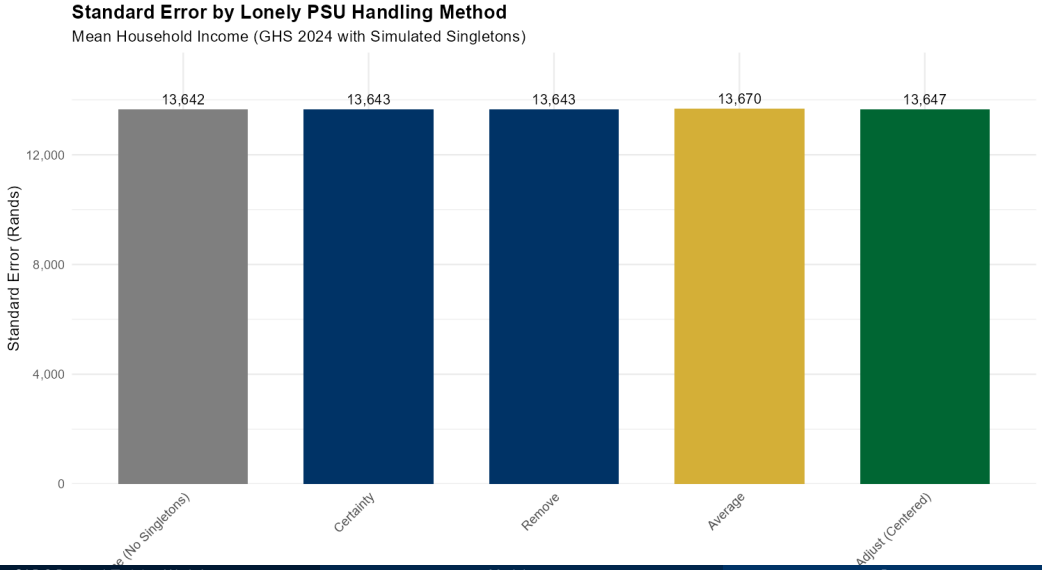
Mean Household Income with 1 Singleton Stratum

Method	Mean (R)	SE (R)	CV (%)	SE Ratio
Baseline (no singleton)	260,166	13,642	5.24	1.0000
Certainty	260,166	13,643	5.24	1.0001
Remove	260,166	13,643	5.24	1.0001
Adjust (Centered)	260,166	13,647	5.24	1.0004
Average	260,166	13,670	5.25	1.0021

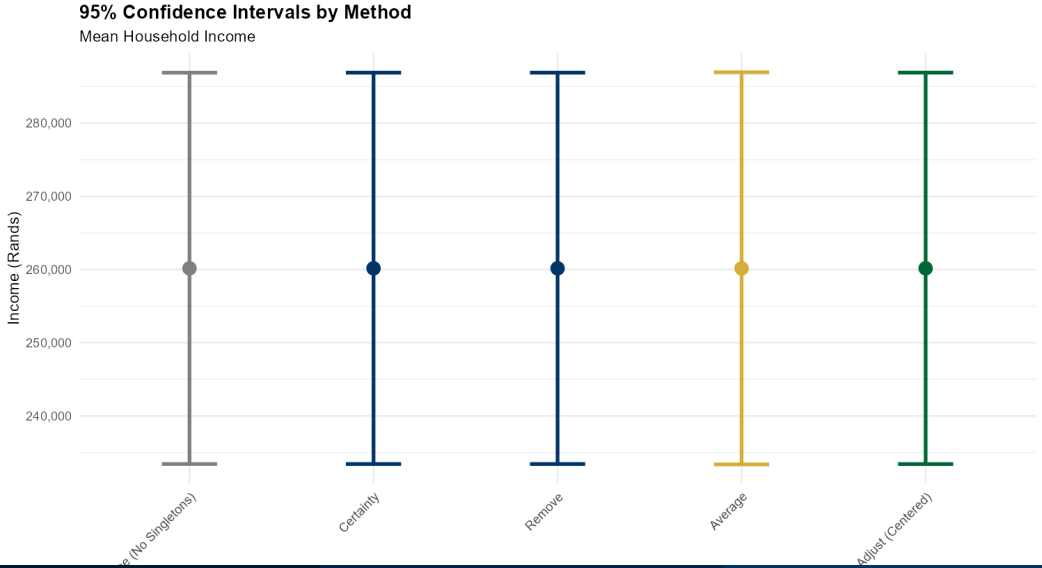
Key observations:

- Point estimate identical across all methods
- SE differences small (R 13,643 – 13,670) because only 1 of 248 strata affected
- Average method gives highest SE (most conservative in this case)

Lab 3.4: Standard Error Comparison



Lab 3.4: Confidence Interval Comparison



When Singleton Impact is Larger

Lab 3.4 showed small impact because:

- Only 1 of 248 strata was singleton
- Singleton stratum had only 3 households
- Large sample dilutes effect

Impact is LARGER when:

1. **Many singletons:** 10+ singleton strata
2. **Large singleton:** Singleton contains significant population share
3. **Extreme values:** Singleton PSU has outlier values
4. **Domain analysis:** Subpopulation has few strata
5. **Rare outcome:** Binary variable with few positives

Rule of thumb: If singletons represent $> 5\%$ of strata, investigate carefully!

Prevention: Avoiding Singleton Strata

At Design Stage:

1. **Sample ≥ 2 PSUs per stratum** — fundamental rule
2. **Collapse small strata** — combine similar strata before sampling
3. **Use implicit stratification** — sort frame, use systematic sampling
4. **Document certainty PSUs** — if unavoidable, note in metadata

At Analysis Stage:

1. **Check data early** — run singleton detection before analysis
2. **Combine strata post-hoc** — merge similar singletons with neighbors
3. **Use replication methods** — may handle singletons more gracefully
4. **Document handling** — report method used in publications

Problem #2: Insufficient Degrees of Freedom

Degrees of freedom for complex surveys:

$$df = \sum_{h=1}^{H} (n_h - 1) = n_{PSU} - H$$

Why df matters:

- Confidence intervals use t_{df} distribution
- Low df $\Rightarrow$ wider critical values $\Rightarrow$ wider CIs
- Very low df (< 30) $\Rightarrow$ unreliable inference

Critical values comparison:

Distribution	df	95% critical value	CI width ratio
Normal (z)	∞	1.96	1.00
t	100	1.98	1.01
t	30	2.04	1.04
t	10	2.23	1.14
t	5	2.57	1.31

When Low Degrees of Freedom Occur

Scenario 1: Few strata, 2 PSUs each

- 10 strata $\times$ 2 PSUs $=$ 20 PSUs
- $df = 20 - 10 = 10$ (problematic!)

Scenario 2: Domain estimation

- National sample: 200 strata, 3,000 PSUs, $df = 2,800$ (fine)
- Small province subsample: 15 strata, 45 PSUs, $df = 30$ (borderline)
- Rare subgroup: 8 strata with observations, $df = 8$ (problematic!)

Scenario 3: Many singleton strata

- Each singleton contributes 0 to df
- 50 strata, 20 singletons: $df = 30 - 0 = 30$ (reduced)

Checking Degrees of Freedom in R

```
library(survey)

# Create design
des <- svydesign(id = ~psu, strata = ~stratum,
                 weights = ~house_wgt, data = ghs)

# Overall degrees of freedom
cat("Total df:", degf(des), "\n")

# For subpopulations, df based on observed strata
des_sub <- subset(des, province == "Northern Bushveld")
cat("Subpopulation df:", degf(des_sub), "\n")

# Detailed breakdown
n_psu <- length(unique(ghs$psu))
n_strata <- length(unique(ghs$stratum))
cat("PSUs:", n_psu, "Strata:", n_strata, "\n")
cat("Calculated df:", n_psu - n_strata, "\n")

# Check if any stratum has few PSUs
psu_per_stratum <- table(unique(ghs[, c("stratum", "psu")])$stratum)
summary(as.numeric(psu_per_stratum))
```

Checking Degrees of Freedom in STATA

```
* Setup survey design
svyset psu [pw=house_wgt], strata(stratum)

* View design summary including df
svydescribe

* After estimation, check df used
svy: mean income
* Look for "Design df = XXX" in output

* Manual calculation
distinct psu
local n_psu = r(ndistinct)

distinct stratum
local n_strata = r(ndistinct)

display "Degrees of freedom: " `n_psu' - `n_strata'

* For subpopulation
svy, subpop(if province == 3): mean income
* df will be lower, based on observed strata
```

Solutions for Low Degrees of Freedom

1. **Use t-distribution** (not z)
   - Most software does this automatically
   - Verify: check if output shows "t" or "z"
2. **Collapse similar strata**
   - Combine geographically adjacent strata
   - Merge strata with similar characteristics
   - Increases df but loses some stratification benefit
3. **Use Satterthwaite approximation**
   - Adjusts df for complex statistics
   - Built into some software functions
4. **Bootstrap confidence intervals**
   - Percentile method doesn't require df
   - More robust for small samples

Collapsing Strata to Increase df

```
# Original: province x urban/rural = 9 x 3 = 27 strata
# Some strata have only 2 PSUs

# Create collapsed strata (province only)
ghs$stratum_collapsed <- ghs$province

# Or: combine urban types
ghs$stratum_collapsed <- interaction(
  ghs$province,
  ifelse(ghs$geotype == "Urban", "Urban", "NonUrban")
)

# Compare df
des_original <- svydesign(id = ~psu, strata = ~stratum,
                          weights = ~house_wgt, data = ghs)
des_collapsed <- svydesign(id = ~psu, strata = ~stratum_collapsed,
                           weights = ~house_wgt, data = ghs)

cat("Original df:", degf(des_original), "\n")
cat("Collapsed df:", degf(des_collapsed), "\n")
```

Trade-off: More df but less efficient estimation (loses stratification gains)

Problem #3: Negative Variance Estimates

Symptom: SE displayed as `NaN`, or warning about negative variance

Causes:

1. **Extreme weights**
   - Very large weight adjustments
   - Calibration to small control totals
2. **High finite population correction**
   - Sampling fraction close to 1
   - FPC overcorrects variance
3. **Taylor linearization issues**
   - Complex ratio estimators
   - Statistics near boundaries (prop $\approx$ 0 or 1)

Solutions:

- Trim extreme weights
- Use replication methods instead of Taylor
- Check for data errors

Variance Diagnostic Checklist

Symptom	Likely Cause	First Action
SE is `NA` or `.`	Singleton stratum	Set lonely PSU option
SE is `NaN`	Negative variance	Check weights, use replication
SE seems too small	Ignored clustering	Verify PSU variable specified
SE seems too large	Ignored stratification	Add stratum variable
SE varies wildly across replicates	Outlier PSU	Check for data errors
CI includes impossible values	Low df + skewness	Use bootstrap percentile CI
Warning about "certainty PSU"	Single large PSU	Verify if certainty intended
Different results R vs STATA	Different defaults	Check lonely PSU settings

Best Practices for Variance Estimation

Before Analysis:

1. Check for singleton strata — fix before estimating
2. Verify PSU and stratum variables are correct
3. Examine weight distribution for extremes
4. Calculate degrees of freedom

During Analysis:

1. Set lonely PSU option explicitly (don't rely on defaults)
2. Use t-distribution for confidence intervals
3. Compare Taylor and replication methods for key estimates
4. Document all variance estimation choices

For Reporting:

1. Report method used (Taylor, JK, Bootstrap)
2. Note any singleton strata and handling method
3. Include degrees of freedom if low (< 50)
4. Provide CVs alongside point estimates

Software Defaults: Know Your Settings

Setting	R (survey)	STATA
Lonely PSU default	`"fail"` (error)	`missing` (NA)
Default variance method	Taylor	Taylor
CI distribution	t	t
FPC default	Not applied	Not applied
Subpop df calculation	Observed strata	Observed strata

Recommendation: Always set options explicitly!

R:

```
options(
  survey.lonely.psu = "adjust"
)
```

STATA:

```
svyset psu [pw=wgt], ///
  strata(str) ///
  singleunit(centered)
```

Module 3.4 Troubleshooting: Key Takeaways

1. **Singleton strata cause variance estimation failure**
   - Detect early with diagnostic checks
   - Use `adjust/centered` for conservative SEs
   - Prevention at design stage is best
2. **Degrees of freedom affect inference**
   - Low df (< 30) widens confidence intervals
   - Collapse strata if necessary
   - Use bootstrap for very low df situations
3. **Lab 3.4 demonstrated:**
   - Small singleton impact when few strata affected
   - All methods gave similar results (SE: R 13,642–13,670)
   - Average method most conservative in this case
4. **Document your choices**
   - Report variance method and lonely PSU handling
   - Enables reproducibility and transparency

Next: Day 4 — Error Mitigation and Quality Assurance

Day 3 – Module 3.5

Auditing the Weights

A Quality Assurance Checklist

Weight Diagnostics • Outlier Detection • Trimming • DEFF

Zambara Case: The Suspicious Estimate

Zambara Case

Lindiwe presents preliminary results to the Minister. "Average household income is R 260,000," she reports. The Minister frowns: "That seems reasonable nationally, but your estimate for Northern Province is R 890,000. That's triple the capital region!"

Lindiwe investigates and finds one household with a weight of 47,000 — representing 47,000 households in the population. That single respondent is a wealthy mining executive whose income is pulling up the entire provincial estimate.

The lesson: Extreme weights can cause **weight explosions** that destabilize estimates and inflate variance.

The Weight Audit Checklist

- ☑ 1. Distribution check: Range, mean, median, CV
- ☑ 2. Outlier detection: Identify extreme weights
- ☑ 3. Domain balance: Weights sum to known totals?
- ☑ 4. DEFF calculation: Design effect reasonable?
- ☑ 5. Trimming decision: Need to cap extreme weights?
- ☑ 6. Documentation: Record all adjustments

Goal: Ensure weights are **fit for purpose** before analysis begins.

What Are "Weight Explosions"?

Definition

A **weight explosion** occurs when a small number of observations have disproportionately large weights, giving them excessive influence on estimates.

Causes of extreme weights:

1. **Non-response adjustments** — cells with very low response rates
2. **Calibration to small controls** — forcing match to rare categories
3. **Design factors** — oversampling followed by inverse weighting
4. **Post-stratification** — thin cells get inflated weights
5. **Cumulative adjustments** — base $\times$ NR $\times$ calibration compounds

Consequences: High variance, unstable estimates, sensitivity to outliers

Check #1: Weight Distribution Metrics

Key statistics to compute:

Metric	Formula	Warning Threshold
Minimum	$\min(w_i)$	< 1 (should be ≥ 1)
Maximum	$\max(w_i)$	Context-dependent
Mean	$\bar{w} = \frac{1}{n}\sum w_i$	—
Median	$w_{(n/2)}$	—
CV of weights	$\frac{SD(w)}{\bar{w}} \times 100$	$> 100\%$
Max/Min ratio	$\frac{\max(w)}{\min(w)}$	> 50
Max/Mean ratio	$\frac{\max(w)}{\bar{w}}$	> 10

Rule of thumb: If CV of weights $> 100\%$ or max/mean > 10, investigate!

Check #1: Weight Distribution in R

```r
# Basic weight statistics
weight_stats <- function(w) {
  data.frame(
    n = length(w),
    min = min(w),
    max = max(w),
    mean = mean(w),
    median = median(w),
    sd = sd(w),
    cv = sd(w) / mean(w) * 100,
    max_min_ratio = max(w) / min(w),
    max_mean_ratio = max(w) / mean(w),
    sum = sum(w)
  )
}

# Apply to GHS weights
weight_stats(ghs$house_wgt)

# By domain (province)
ghs %>%
  group_by(province) %>%
  summarise(across(house_wgt, list(
    min = min, max = max, cv = ~sd(.)/mean(.)*100
  )))
```

Check #1: Weight Distribution in STATA

```
* Basic weight statistics
summarize house_wgt, detail

* Calculate CV manually
quietly summarize house_wgt
local cv = r(sd) / r(mean) * 100
display "CV of weights: " %5.1f `cv' "%"

* Max/mean ratio
local max_mean = r(max) / r(mean)
display "Max/Mean ratio: " %5.1f `max_mean'

* By province
bysort province: summarize house_wgt, detail

* Tabulate weight distribution
tabstat house_wgt, by(province) ///
    stat(n min max mean sd) columns(statistics)

* Check for suspicious weights
list hhid province house_wgt if house_wgt > 10000, noobs
```

Visualizing Weight Distribution

R Code:

```
library(ggplot2)

# Histogram
ggplot(ghs, aes(x = house_wgt)) +
  geom_histogram(bins = 50,
                 fill = "steelblue") +
  geom_vline(xintercept =
    mean(ghs$house_wgt),
    color = "red", lwd = 1) +
  scale_x_log10() +
  labs(title = "Weight Distribution",
       x = "Weight (log scale)")

# Boxplot by domain
ggplot(ghs, aes(x = province,
                y = house_wgt)) +
  geom_boxplot() +
  coord_flip()
```

What to look for:

- Long right tail → extreme weights
- Outliers beyond whiskers
- Domain differences
- Bimodal distribution (design issue?)

Log scale helps visualize when weights span several orders of magnitude.

Check #2: Identifying Outlier Weights

Methods for detecting extreme weights:

1. **Fixed threshold**
   - Flag if $w_i > k \times \bar{w}$ (e.g., $k = 5$ or 10)
   - Simple but arbitrary
2. **Interquartile range (IQR)**
   - Flag if $w_i > Q_3 + 1.5 \times IQR$ (standard boxplot rule)
   - Or use $3 \times IQR$ for extreme outliers
3. **Percentile-based**
   - Flag weights above 99th or 99.5th percentile
   - Data-driven, robust to distribution shape
4. **Domain-specific**
   - Apply thresholds within each stratum/domain
   - Accounts for designed weight variation

Check #2: Outlier Detection Code

```
# Method 1: Multiple of mean
threshold_mean <- 5 * mean(ghs$house_wgt)
outliers_mean <- ghs %>% filter(house_wgt > threshold_mean)
cat("Outliers␣(>5x␣mean):", nrow(outliers_mean), "\n")

# Method 2: IQR rule
Q1 <- quantile(ghs$house_wgt, 0.25)
Q3 <- quantile(ghs$house_wgt, 0.75)
IQR_val <- Q3 - Q1
threshold_iqr <- Q3 + 3 * IQR_val  # Extreme outliers
outliers_iqr <- ghs %>% filter(house_wgt > threshold_iqr)
cat("Outliers␣(>Q3+3*IQR):", nrow(outliers_iqr), "\n")

# Method 3: Percentile
threshold_pct <- quantile(ghs$house_wgt, 0.995)
outliers_pct <- ghs %>% filter(house_wgt > threshold_pct)
cat("Outliers␣(>99.5th␣pctl):", nrow(outliers_pct), "\n")

# Examine outliers
outliers_pct %>%
  select(hhid, province, stratum, house_wgt, totmhinc) %>%
  arrange(desc(house_wgt))
```

Why Trim Extreme Weights?

The bias-variance tradeoff:

Without trimming:

- Weights are "correct" (unbiased)
- But variance can be huge
- Single outlier dominates estimate
- Results unstable across samples

With trimming:

- Introduces small bias
- But variance reduced substantially
- MSE often lower overall
- Results more stable

Mean Squared Error:

$$MSE = Bias^2 + Variance$$

Trimming *increases* bias but *decreases* variance. Often $MSE_{trimmed} < MSE_{untrimmed}$.

Weight Trimming Methods

Method	Description	Pros/Cons
Hard cap	Set $w_i = \min(w_i, c)$	Simple; discontinuity at cap
Percentile cap	Cap at pth percentile	Data-driven; still discontinuous
Winsorization	Replace extremes with boundary values	Preserves ranks
Weight smoothing	Redistribute excess to similar units	Maintains totals
Iterative	Trim then recalibrate	Totals preserved; more complex

Most common: Percentile cap (e.g., cap at 99th percentile)

Trimming Weights in R

```
# Method 1: Hard cap at fixed value
cap_value <- 5000
ghs$wgt_capped <- pmin(ghs$house_wgt, cap_value)

# Method 2: Cap at 99th percentile
pct99 <- quantile(ghs$house_wgt, 0.99)
ghs$wgt_pct99 <- pmin(ghs$house_wgt, pct99)

# Method 3: Cap at k times median
k <- 6
cap_median <- k * median(ghs$house_wgt)
ghs$wgt_median_cap <- pmin(ghs$house_wgt, cap_median)

# Compare distributions
summary(ghs$house_wgt)      # Original
summary(ghs$wgt_capped)     # Hard cap
summary(ghs$wgt_pct99)      # Percentile cap

# Check how many trimmed
cat("Trimmed at 99th pctl:",
    sum(ghs$house_wgt > pct99), "households\n")
cat("Weight lost:",
    sum(ghs$house_wgt) - sum(ghs$wgt_pct99), "\n")
```

Trimming Weights in STATA

```
* Method 1: Hard cap
gen wgt_capped = min(house_wgt, 5000)

* Method 2: Cap at 99th percentile
quietly summarize house_wgt, detail
local pct99 = r(p99)
gen wgt_pct99 = min(house_wgt, `pct99')

* Method 3: Cap at k times median
local k = 6
quietly summarize house_wgt, detail
local cap = `k' * r(p50)
gen wgt_median_cap = min(house_wgt, `cap')

* Compare
summarize house_wgt wgt_capped wgt_pct99 wgt_median_cap

* Count trimmed observations
count if house_wgt > `pct99'
display "Observations trimmed: " r(N)

* Weight sum comparison (check loss)
total house_wgt wgt_pct99
```

Rescaling Trimmed Weights

Problem: After trimming, weights no longer sum to population total.

Solution: Rescale to preserve control totals.

```
# Original population total (from weights)
pop_total <- sum(ghs$house_wgt)

# Trimmed weights sum
trimmed_sum <- sum(ghs$wgt_pct99)

# Rescale to original total
ghs$wgt_trimmed_scaled <- ghs$wgt_pct99 * (pop_total / trimmed_sum)

# Verify
cat("Original sum:", pop_total, "\n")
cat("Trimmed sum:", trimmed_sum, "\n")
cat("Scaled sum:", sum(ghs$wgt_trimmed_scaled), "\n")

# Alternative: Redistribute excess proportionally
excess <- sum(ghs$house_wgt - ghs$wgt_pct99)
ghs$wgt_redistributed <- ghs$wgt_pct99 +
  (ghs$wgt_pct99 / trimmed_sum) * excess
```

Analyzing Trimming Impact

Before finalizing trimmed weights, compare:

Diagnostic	Original	Trimmed
Weight CV (%)	Calculate	Should decrease
Max/Mean ratio	Calculate	Should decrease
Mean estimate	$\hat{\bar{y}}_{orig}$	$\hat{\bar{y}}_{trim}$
SE of mean	SE_{orig}	Should decrease
DEFF	$DEFF_{orig}$	Should decrease
Domain estimates	Check each	Sensitivity?

Key question: Does the reduction in variance justify the introduced bias?

Sensitivity analysis: Try multiple trimming thresholds (95th, 99th, 99.5th)

Check #3: Domain Balance

Weights should sum to known population totals by domain:

Province	Weight Sum	Census Pop	Ratio
Capital Region	4,512,000	4,500,000	1.003
Eastern Highlands	3,890,000	3,900,000	0.997
Northern Bushveld	2,105,000	2,100,000	1.002
⋮	⋮	⋮	⋮
National	**18,750,000**	**18,750,000**	**1.000**

Expected: Ratio ≈ 1.00 for calibrated weights

Red flag: Ratios > 1.05 or < 0.95 suggest calibration problems

Check #3: Domain Balance Code

```
# Calculate weight sums by province
weight_sums <- ghs %>%
  group_by(province) %>%
  summarise(
    n_obs = n(),
    weight_sum = sum(house_wgt),
    .groups = "drop"
  )

# Add known population totals (from census/admin data)
census_pop <- data.frame(
  province = c("Gauteng", "KwaZulu-Natal", "Western␣Cape", ...),
  pop_total = c(4500000, 3900000, 2800000, ...)
)

# Merge and calculate ratio
weight_sums <- weight_sums %>%
  left_join(census_pop, by = "province") %>%
  mutate(ratio = weight_sum / pop_total)

# Display
print(weight_sums)

# Flag problematic domains
weight_sums %>% filter(ratio < 0.95 | ratio > 1.05)
```

Check #4: Design Effect (DEFF)

Definition

The **Design Effect (DEFF)** measures how much larger the variance of a complex survey estimate is compared to a simple random sample of the same size.

$$DEFF = \frac{Var_{complex}(\hat{\theta})}{Var_{SRS}(\hat{\theta})} = \frac{Var_{complex}}{s^2/n}$$

Interpretation:

- $DEFF = 1$: Complex design as efficient as SRS
- $DEFF = 2$: Need 2x sample size for same precision as SRS
- $DEFF > 4$: Investigate — possibly problematic design or weights

Components of Design Effect

DEFF can be decomposed:

$$DEFF \approx DEFF_{cluster} \times DEFF_{strat} \times DEFF_{weight}$$

1. **Clustering effect** $(DEFF_{cluster})$
   - $DEFF_{cluster} \approx 1 + (b-1)\rho$
   - $b =$ average cluster size, $\rho =$ intra-class correlation
   - Usually **increases** DEFF
2. **Stratification effect** $(DEFF_{strat})$
   - Usually < 1 (stratification **reduces** variance)
3. **Weighting effect** $(DEFF_{weight})$
   - $DEFF_{weight} \approx 1 + CV(w)^2$
   - Variable weights **increase** DEFF

DEFF Due to Unequal Weighting

Kish's approximation for weighting effect:

$$DEFF_{weight} = \frac{n \sum w_i^2}{(\sum w_i)^2} = 1 + CV(w)^2$$

Examples:

Weight CV	$DEFF_{weight}$	**Effective sample loss**
0% (equal)	1.00	0%
50%	1.25	20%
100%	2.00	50%
150%	3.25	69%
200%	5.00	80%

Key insight: Weight CV $> 100\%$ means you lose $> 50\%$ of effective sample size!

Calculating DEFF in R

```
library(survey)

# Create survey design
des <- svydesign(id = ~psu, strata = ~stratum,
                 weights = ~house_wgt, data = ghs)

# Method 1: From svymean output
mean_result <- svymean(~totmhinc, des, na.rm = TRUE, deff = TRUE)
print(mean_result)
# DEFF shown in output

# Method 2: Extract DEFF explicitly
deff_val <- deff(mean_result)
cat("DEFF:", deff_val, "\n")

# Method 3: Kish's approximation (weighting effect only)
n <- nrow(ghs)
w <- ghs$house_wgt
deff_kish <- n * sum(w^2) / sum(w)^2
cat("Kish DEFF (weight only):", deff_kish, "\n")

# Effective sample size
n_eff <- n / deff_val
cat("Effective sample size:", round(n_eff, 0), "\n")
```

Calculating DEFF in STATA

```
* Setup survey design
svyset psu [pw=house_wgt], strata(stratum)

* Method 1: Request DEFF in estimation
svy: mean totmhinc
estat effects
* Displays DEFF, DEFT, and MEFF

* Method 2: Kish approximation
quietly summarize house_wgt
local n = r(N)
gen wgt_sq = house_wgt^2
quietly summarize wgt_sq
local sum_w_sq = r(sum)
quietly summarize house_wgt
local sum_w = r(sum)

local deff_kish = `n' * `sum_w_sq' / (`sum_w'^2)
display "Kish DEFF: " %5.3f `deff_kish'

* Effective sample size
display "Effective n: " %8.0f `n' / `deff_kish'

drop wgt_sq
```

DEFF by Domain

```
# Calculate DEFF for each province
deff_by_province <- ghs %>%
  group_by(province) %>%
  summarise(
    n = n(),
    cv_weights = sd(house_wgt) / mean(house_wgt) * 100,
    deff_kish = n() * sum(house_wgt^2) / sum(house_wgt)^2,
    n_effective = n() / deff_kish,
    .groups = "drop"
  ) %>%
  arrange(desc(deff_kish))

print(deff_by_province)
```

What to look for:

- Domains with $DEFF > 3$ — investigate weight distribution
- Large variation across domains — design imbalance?
- Domains with very low effective n — unreliable estimates

Using DEFF as a Quality Threshold

DEFF benchmarks:

DEFF Range	Rating	Action
$1.0 - 1.5$	Excellent	No action needed
$1.5 - 2.5$	Good	Typical for cluster surveys
$2.5 - 4.0$	Moderate	Review design, consider trimming
$4.0 - 6.0$	Concerning	Investigate weights, likely trim
> 6.0	Problematic	Major review required

Context matters:

- High clustering (ρ) naturally increases DEFF
- Oversampling designs have higher expected DEFF
- Compare to planning DEFF from design documentation

Effective Sample Size

Convert DEFF to actionable metric:

$$n_{eff} = \frac{n}{DEFF}$$

Example: GHS 2024 with $n = 20,940$ and $DEFF = 17$

$$n_{eff} = \frac{20,940}{17} \approx 1,232$$

Interpretation: Despite interviewing 20,940 households, precision is equivalent to an SRS of only 1,232 households!

For subgroups:

- Province with $n = 1,500$ and $DEFF = 3$: $n_{eff} = 500$
- Rare subgroup with $n = 200$ and $DEFF = 2$: $n_{eff} = 100$
- Reliability threshold: $n_{eff} \geq 50$ for publishable estimates

Complete Weight Audit Checklist

	Check	Threshold	Pass?
1	All weights > 0	$\min(w) > 0$	☐
2	Weight CV reasonable	$CV(w) < 100\%$	☐
3	Max/Mean ratio acceptable	< 10	☐
4	No extreme outliers	$< 0.5\%$ above 99th pctl	☐
5	Domain sums match controls	Ratio $0.95 - 1.05$	☐
6	DEFF within expectations	< 4 (or design target)	☐
7	Effective n sufficient	$n_{eff} \geq 50$ per domain	☐
8	Trimming documented	If applied	☐
9	Rescaling documented	If trimmed	☐
10	Final weights verified	Sum to population	☐

Sign-off: All checks passed before proceeding to analysis!

Check #6: Documentation Requirements

For each weighting step, document:

1. **Base weights**
   - Selection probabilities at each stage
   - Formula and source data
2. **Non-response adjustments**
   - Adjustment cells and response rates
   - Method (weighting class, propensity)
3. **Calibration**
   - Control totals and sources
   - Calibration variables and method
4. **Trimming**
   - Threshold used and justification
   - Number of weights trimmed
   - Rescaling method
5. **Final diagnostics**
   - Weight distribution statistics
   - DEFF by key domains

Common Weight Problems and Solutions

Problem	Cause	Solution
Very high CV ($> 150\%$)	Extreme non-response adjustment	Collapse adjustment cells
Negative weights	Calibration error	Check constraints, use bounded methods
Weights don't sum to total	Trimming without rescaling	Rescale proportionally
DEFF varies wildly by domain	Design imbalance	Review allocation, consider pooling
Single obs dominates domain	Outlier weight + outlier value	Trim weight AND/OR value
Bimodal weight distribution	Mixed design elements	Investigate, may be intentional

Module 3.5: Key Takeaways

1. **Always audit weights before analysis**
   - Check distribution: min, max, CV, outliers
   - Verify domain totals match controls
2. **Watch for weight explosions**
   - CV $> 100\%$ is a warning sign
   - Max/Mean > 10 requires investigation
3. **Trimming involves bias-variance tradeoff**
   - Small bias, potentially large variance reduction
   - Always rescale after trimming
   - Document threshold and justification
4. **Use DEFF as quality indicator**
   - $DEFF > 4$ warrants investigation
   - Calculate effective sample size: $n_{eff} = n/DEFF$
   - Ensure $n_{eff} \geq 50$ for publishable domains

Day 3 Complete! Next: Day 4 — Error Mitigation

Day 3 – Synthesis

Bringing It All Together

The Complete Weighting Pipeline

Recap • Homework • Day 4 Preview

Zambara Case: Lindiwe's Day 3 Journey

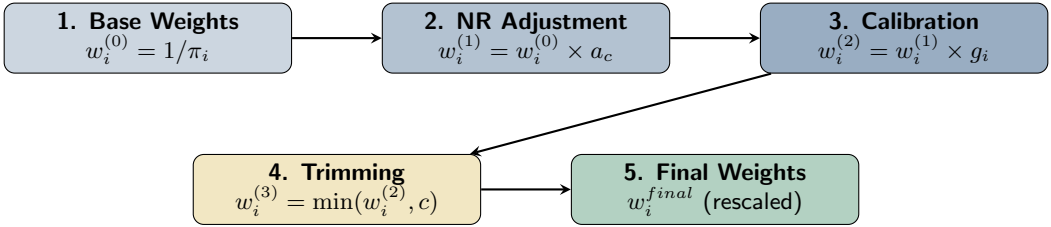
Zambara Case

Lindiwe started the day with raw survey data and selection probabilities. By day's end, she has:

- Calculated base weights from design
- Adjusted for 18% unit non-response
- Calibrated to provincial population totals
- Trimmed 12 extreme weights at the 99th percentile
- Verified DEFF = 2.3 (acceptable for cluster design)
- Documented every step for reproducibility

"The weights are ready. Now I can produce defensible estimates!"

The Complete Weighting Pipeline



Each stage: Documented, verified, and quality-checked before proceeding.

Stage 1: Base Weights Recap

Key Concepts:

- Inverse of selection probability
- Multi-stage: product of stage probabilities
- Each unit "represents" w_i population units

Formula:

$$w_i^{base} = \frac{1}{\pi_i} = \frac{1}{\pi_{1i} \times \pi_{2i|1} \times \pi_{3i|2}}$$

Quality Checks:

1. All $w_i > 0$?
2. $\sum w_i \approx N$ (frame size)?
3. Reasonable range?
4. No coding errors?

Common Issues:

- Missing MOS data
- Stage probability errors
- Frame coverage gaps

Stage 2: Non-Response Adjustment Recap

Key Concepts:

- Compensate for unit non-response
- Assumes MAR within adjustment cells
- Inflate respondent weights

Methods Covered:

1. Weighting class adjustment
2. Response propensity weighting
3. CHAID-based cells

Formula:

$$a_c = \frac{\sum_{i \in c} w_i^{base}}{\sum_{i \in c,R} w_i^{base}}$$

Quality Checks:

1. Response rates by cell
2. Adjustment factor range
3. Cell sizes adequate?
4. Weight CV after adjustment

Warning Signs:

- $a_c > 3$ (very low RR)
- Cells with $n < 20$
- Extreme weight inflation

Stage 3: Calibration Recap

Key Concepts:

- Force weighted totals to match known controls
- Reduces coverage and sampling error
- Multiple methods available

Methods Covered:

1. Post-stratification
2. Raking (iterative proportional fitting)
3. GREG (generalized regression)

Constraint:

$$\sum_{i \in s} w_i^{cal} \cdot x_i = X \quad \text{(known total)}$$

Quality Checks:

1. Convergence achieved?
2. All constraints met?
3. g-weight range acceptable?
4. Weight CV after calibration

Warning Signs:

- Non-convergence
- $g < 0.3$ or $g > 3$
- Control totals inconsistent

Stage 4: Weight Trimming Recap

Key Concepts:

- Cap extreme weights to reduce variance
- Trade-off: small bias for large variance reduction
- Must rescale to preserve totals

Methods Covered:

1. Hard cap (fixed threshold)
2. Percentile cap (99th, 99.5th)
3. Multiple of median

Rescaling:

$$w_i^{rescaled} = w_i^{trimmed} \times \frac{\sum w^{original}}{\sum w^{trimmed}}$$

Quality Checks:

1. How many trimmed?
2. Impact on key estimates?
3. Domain effects?
4. Totals preserved?

Decision Criteria:

- Trim if CV $> 100\%$
- Trim if max/mean > 10
- Document threshold choice

Stage 5: Final Quality Audit Recap

The 10-Point Checklist:

1. All weights > 0
2. Weight CV $< 100\%$
3. Max/Mean ratio < 10
4. Outliers $< 0.5\%$ of sample
5. Domain sums match controls
6. DEFF within target
7. $n_{eff} \geq 50$ per domain
8. Trimming documented
9. Rescaling documented
10. Final sum = population

Key Metrics:

Metric	Good	Acceptable	Investigate
Weight CV	$< 50\%$	$50 - 100\%$	$> 100\%$
DEFF	< 2	$2 - 4$	> 4
Max/Mean	< 5	$5 - 10$	> 10

Variance Estimation Recap

Two Families of Methods:

Taylor Linearization:

- Analytical approximation
- Fast computation
- Built into standard software
- Best for means, totals, ratios

Use when:

- Standard statistics
- Large samples
- Need speed

Replication Methods:

- Jackknife, BRR, Bootstrap
- Works for any statistic
- Handles complex designs
- Enables public-use files

Use when:

- Medians, percentiles
- Complex statistics
- Need percentile CIs

Critical: Handle lonely PSUs with `adjust/centered` option!

Day 3 Learning Objectives: Self-Assessment

Can you now...

- ☐ Calculate base weights from multi-stage selection probabilities?
- ☐ Implement non-response weight adjustments using weighting classes?
- ☐ Apply calibration (raking, GREG) in R and STATA?
- ☐ Estimate variance using Taylor linearization?
- ☐ Implement replication methods (Jackknife, Bootstrap)?
- ☐ Diagnose and fix lonely PSU problems?
- ☐ Detect and trim extreme weights?
- ☐ Use DEFF as a quality indicator?
- ☐ Document the complete weighting pipeline?

If any boxes unchecked: Review relevant module before homework!

Day 3: Key Formulas Reference Card

Concept	Formula
Base weight	$w_i = 1/\pi_i$
NR adjustment	$a_c = \sum w_i^{base} / \sum_R w_i^{base}$
Raking g-weight	$g_i = \prod_k \lambda_k^{x_{ik}}$ (iterative)
GREG weight	$g_i = 1 + (X - \hat{X})^T \hat{T}^{-1} x_i$
Taylor variance	$\widehat{Var}(\hat{\theta}) \approx \sum_h \frac{n_h}{n_h - 1} \sum_i (e_{hi} - \bar{e}_h)^2$
Jackknife variance	$\widehat{Var}_{JK} = \sum_h \frac{n_h - 1}{n_h} \sum_i (\hat{\theta}_{(hi)} - \hat{\theta})^2$
Bootstrap variance	$\widehat{Var}_{Boot} = \frac{1}{B-1} \sum_b (\hat{\theta}^{(b)} - \bar{\theta}^*)^2$
DEFF (weighting)	$DEFF_w = 1 + CV(w)^2$
Effective n	$n_{eff} = n/DEFF$

Day 3: Software Commands Reference

R (survey package):

```r
# Survey design
svydesign(id=~psu,
  strata=~stratum,
  weights=~wgt, data=df)

# Calibration
calibrate(des, formula,
  population, calfun="raking")

# Variance
svymean(~y, des, deff=TRUE)

# Lonely PSU
options(survey.lonely.psu
  ="adjust")

# Replication
as.svrepdesign(des,
  type="JKn")
```

STATA:

```stata
* Survey design
svyset psu [pw=wgt], ///
  strata(stratum) ///
  singleunit(centered)

* Estimation
svy: mean y
svy: total y

* Design effects
estat effects

* Calibration
ipfraking [pw=wgt], ///
  ctotal(var pop) ///
  generate(wgt_cal)

* Replication weights
bsweights bsw, n(500)
```

Tonight's Homework: "The Calibrator"

Assignment Overview

You will receive a **toy dataset** representing the Zambara Mini-Survey:

- 500 households across 5 provinces
- 50 PSUs (10 per province)
- Variables: income, employment, household size
- Known population totals provided

Your task: Complete the full weighting pipeline!

Time estimate: 60–90 minutes
Tools: R or STATA (your choice)
Deliverable: Completed script + summary report

Homework Dataset: Zambara Mini-Survey

Dataset structure:

Variable	Type	Description
`hhid`	ID	Household identifier
`psu`	Factor	PSU cluster (1–50)
`province`	Factor	Province (A, B, C, D, E)
`urban`	Binary	Urban (1) vs Rural (0)
`prob_psu`	Numeric	PSU selection probability
`prob_hh`	Numeric	HH selection probability
`responded`	Binary	Response status (1=yes)
`income`	Numeric	Monthly income (Rands)
`employed`	Binary	Employment status
`hhsize`	Integer	Household size

Note: 85% response rate overall (varies by province)

Homework: Known Population Controls

From Zambara Census 2024:

Province	**Households**	**Urban HH**	**Rural HH**
Province A	120,000	85,000	35,000
Province B	95,000	55,000	40,000
Province C	80,000	30,000	50,000
Province D	65,000	25,000	40,000
Province E	40,000	15,000	25,000
Total	**400,000**	**210,000**	**190,000**

Use these totals for calibration in Task 3!

Homework Task 1: Base Weights

Task 1: Calculate Base Weights (15 min)

1. Load the dataset (`zambara_mini.csv` or `.dta`)
2. Calculate base weights: $w^{base} = \frac{1}{prob_psu \times prob_hh}$
3. Verify: $\sum w^{base} \approx$ frame size (450,000 eligible HH)
4. Report: min, max, mean, CV of base weights

Expected output:

- New variable: `wgt_base`
- Summary statistics table
- Histogram of base weight distribution

Homework Task 2: Non-Response Adjustment

Task 2: Adjust for Non-Response (20 min)

1. Calculate response rates by province $\times$ urban
2. Create adjustment cells (province $\times$ urban $=$ 10 cells)
3. Calculate adjustment factors: $a_c = \frac{\sum w_{all}^{base}}{\sum w_{resp}^{base}}$
4. Apply: $w^{NR} = w^{base} \times a_c$ (respondents only)
5. Report: adjustment factors by cell, weight CV after adjustment

Expected output:

- Table of response rates and adjustment factors
- New variable: `wgt_nr`
- Comparison: CV before vs after NR adjustment

Homework Task 3: Calibration

Task 3: Calibrate to Population Controls (25 min)

1. Set up calibration targets: province $\times$ urban totals
2. Apply raking calibration (or post-stratification)
3. Verify: weighted totals match controls exactly
4. Report: g-weight distribution (min, max, mean)
5. Check: any $g < 0.3$ or $g > 3$?

R hint: Use `calibrate()` from survey package
STATA hint: Use `ipfraking` (install if needed)

Expected output:

- New variable: `wgt_cal`
- Verification table: weighted sums vs controls

Homework Task 4: Quality Audit

Task 4: Audit Final Weights (15 min)

1. Calculate weight diagnostics:
   - CV of calibrated weights
   - Max/Mean ratio
   - Identify outliers (above 99th percentile)
2. Decide: Is trimming needed? If yes, trim at 99th percentile
3. Calculate DEFF for mean income estimate
4. Report effective sample size by province

Expected output:

- Weight audit checklist (10 items)
- Final variable: `wgt_final`
- DEFF and n_{eff} table by province

Homework Task 5: Estimation & Comparison

Task 5: Produce Final Estimates (15 min)

1. Create survey design object with final weights
2. Estimate mean household income (national and by province)
3. Compare estimates using:
   - No weights (naive)
   - Base weights only
   - Final calibrated weights
4. Calculate SEs using Taylor linearization
5. Report: How much do estimates differ? Why?

Expected output:

- Comparison table: estimate, SE, CV by weighting approach
- Brief interpretation (3–5 sentences)

Homework Deliverables

Submit by 8:00 AM tomorrow:

1. **Script file** (`.R` or `.do`)
   - Well-commented code
   - Clearly labeled sections for each task
   - Reproducible (runs without errors)
2. **Summary report** (1–2 pages, Word/PDF)
   - Key statistics at each pipeline stage
   - Final weight diagnostics
   - Comparison of weighted vs unweighted estimates
   - One insight or lesson learned
3. **Output dataset** (`.csv` or `.dta`)
   - Include all weight variables created
   - `wgt_base`, `wgt_nr`, `wgt_cal`, `wgt_final`

Homework Grading Rubric

Criterion	Points	Earned
Task 1: Base weights correct	15	
Task 2: NR adjustment implemented	20	
Task 3: Calibration converges, matches controls	25	
Task 4: Quality audit complete	15	
Task 5: Estimates compared, interpreted	15	
Code quality & documentation	10	
Total	**100**	

Bonus (5 points): Compare Taylor vs Jackknife SE for mean income

Homework Hints

Common pitfalls to avoid:

1. **Forgetting to subset respondents**
   - NR-adjusted weights only apply to respondents
   - Filter: `responded == 1` after Task 2
2. **Calibration targets don't match data categories**
   - Ensure province/urban coding matches control table
   - Check for missing values
3. **Not rescaling after trimming**
   - Trimmed weights must sum to population total
4. **Forgetting lonely PSU setting**

```
# R
options(survey.lonely.psu = "adjust")
```

Coming Tomorrow

Day 4

Error Mitigation & Quality Assurance

Total Survey Error • Non-Response • Imputation • Fragile Settings

Day 4 Preview: Total Survey Error Framework

Error Sources

- Coverage Error
- Sampling Error
- Non-Response Error
- Measurement Error
- Processing Error

→ **Total Survey Error**

Key insight: Sampling error is just *one* component of total error. A well-designed survey minimizes *all* error sources!

Day 4 Preview: Module Overview

Module	Topic	Slides
4.1	Total Survey Error Framework	1–15
4.2	Non-Response Analysis & Diagnosis	16–30
4.3	Imputation Methods	31–45
4.4	Surveys in Fragile Settings	46–55
4.5	Quality Assurance Integration	56–70

Lab exercises:

- Lab 4.2: Response propensity modeling
- Lab 4.3: Hot-deck imputation implementation
- Lab 4.4: Adaptive design simulation
- Lab 4.5: QA checklist application

Day 4 Preview: Key Questions We'll Answer

1. **How do we measure total survey quality?**
   - Beyond sampling error: the full error taxonomy
2. **When is non-response bias serious?**
   - R-indicators and bias diagnostics
3. **How do we handle missing data?**
   - Single vs multiple imputation
   - Hot-deck, regression, predictive mean matching
4. **What do we do when fieldwork fails?**
   - Conflict zones, natural disasters, access restrictions
   - Adaptive and responsive designs
5. **How do we build quality into the process?**
   - Paradata analysis, real-time monitoring

Zambara Case Preview: Day 4

Coming Tomorrow

Lindiwe's weighted estimates are ready, but problems emerge:

- Non-response in **Northern Province** exceeds 40%
- Three PSUs in **Eastern District** were inaccessible due to flooding
- Item non-response on **income** reaches 25% in urban areas
- Field supervisors report **suspicious patterns** in interviewer data

How will Lindiwe diagnose these problems, implement corrections, and ensure the final estimates are fit for purpose?

Day 4 theme: When things go wrong — and how to fix them!

Preparing for Day 4

Before tomorrow morning:

1. **Complete homework** ("The Calibrator")
   - Submit by 8:00 AM
   - Be ready to discuss challenges
2. **Review Day 3 materials**
   - Weighting pipeline stages
   - Variance estimation methods
3. **Optional reading**
   - Groves et al. (2009), Ch. 2: "Inference and Error in Surveys"
   - Lohr (2022), Ch. 8: "Nonresponse"
4. **Bring questions!**
   - What weighting challenges have you faced?
   - What quality issues concern you most?

Day 3 Complete!

Today we covered:

- ✔ Base weights from selection probabilities
- ✔ Non-response adjustments
- ✔ Calibration (raking, GREG)
- ✔ Variance estimation (Taylor, replication)
- ✔ Weight quality auditing

Homework due: Tomorrow 8:00 AM
Day 4 starts: 9:00 AM — Total Survey Error

Questions? Find the instructors during the break!